

Computer Science Internal Assessment



Caribbean Advanced Proficiency Internal Assessment Project for Computer Science, Unit 2

Candidate Name: Robyn Kejana-Jade Sterling

Table of Contents

Specification of Requirements	3
Background	
Problem Context.....	
Problem Definition	
Problem Objectives	
Problem Suited SDLC Model	
Technique Analysis	
Data Flow Diagram: Context Level	7
Data Flow Diagram: Level 1	8
Entity Relationship Diagram	9
Requirements	11
Functional Requirements.....	
Nonfunctional requirements.....	
Design Specifications - System Structure	13
HIPO Chart	14
User Interface	16
Technical Manual.....	28
Algorithm Design – Pseudocode	59
Data Structures and Record	86
Source Code – C Code	88
Test Plan	110
Test Results	119
Error Trapping.....	136
Correct Output.....	139
Usability	141
Conclusion	143
Appendix	144

Problem Definition

Background



Amethyst Accueil is a proficiently active firm, though being relatively small, that distributes a variety of crystals and herbs to the public. In recent times, they've started to break through in the market, which has brought them many changes subsequent from opportunities. They established in the late 2021 but began full operation in early 2022. The firm operates within the botanical parish of St. Ann in Jamaica and is family-owned under

the Paulichi lineage, to which its location sits at the Mahoe Avenue within that of St. Ann.

Problem Context

In the firm's digital-based system, they employ a sophisticated payroll management program that accurately calculates compensation for each employee, while accounting for distinct details like their tax levies, overtime contributions and such. Simultaneously, this system generates comprehensive financial reports, providing insights into revenue, expenses, and profits. The firm has also invested in securing these financial documents in the system vault, as well as issuing e-copies to those of whom it may concern, whether internally or externally.

Additionally, they maintain an employee filing system within the program, allowing seamless

additions, removals, searches, views, and updates. This streamlined approach enhances efficiency, transparency, and effective management of employee records.

Problem Definition

However, the current program exhibits limitations in its functionality, particularly in managing employee records. There is a constraint that restricts the addition of more than six employees to the system, hindering scalability for businesses with larger staff sizes. Additionally, the delete record option appears to be encountering issues, creating potential data management challenges. Furthermore, the linking of pay to employees is nonexistent, posing a significant obstacle in efficiently managing payroll processes. Accessing financial reports and payroll information is also cumbersome, as users are compelled to navigate through the entire system rather than having a streamlined and direct access method.

Problem Objectives

To address the program's limitations, immediate steps should include expanding employee capacity, resolving delete record issues, implementing a direct pay-employee link, and improving system accessibility for streamlined financial reporting and payroll management. Additionally, maintain the update, search, and display functions to match the structure of fixing delete and add records. Create separate files for reports and payrolls to enhance organization and implement

date tracking to keep records of time created, ensuring thorough audit trails and efficient management.

SDLC Model

The Agile Model is ideal for Amethyst Accueil due to its flexibility and adaptability. With limited tech knowledge and urgent issues like imbalanced wages and inaccurate records, Agile enables rapid, incremental development. It fosters collaboration, transparency, and iterative refinement based on user feedback, prioritizing critical issues like payroll accuracy early on. Two major advantages of Agile are its flexibility and rapid delivery, accommodating changes even late in development. However, scope creep may prolong schedules and increase budgets. Maintaining an Agile system requires highly skilled individuals, potentially increasing costs. Without skilled personnel, Agile's effectiveness may be compromised.

Technique Analysis

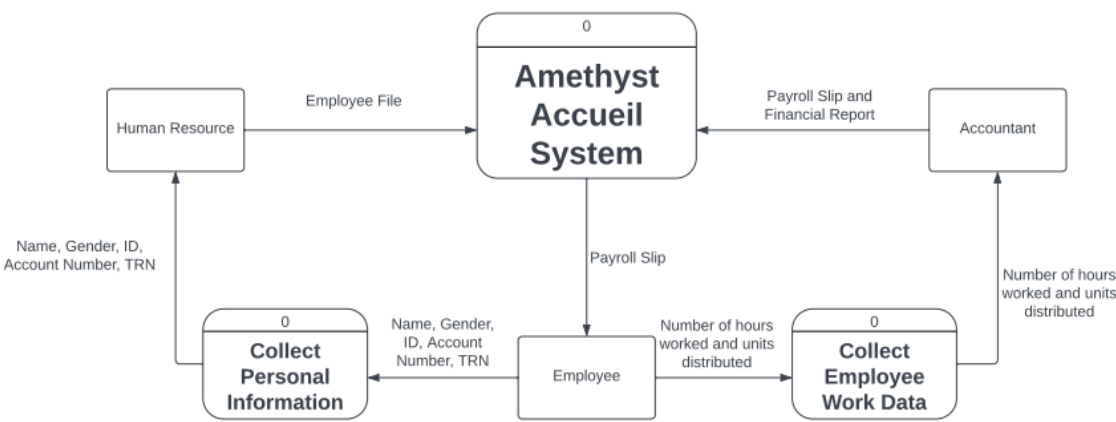
In assessing the efficiency and effectiveness of Amethyst Accueil objectively, various data collection methods were employed to gather feedback on the system's conditions, mainly interviews and questionnaires.

An interview is a real-time conversation between an interviewer and an interviewee. In this case, a structured interview was used, involving predetermined questions to gather in-depth information about the firm's challenges and aspirations. The purpose is to understand stakeholders' expectations and aspirations. Benefits include qualitative insights to address issues

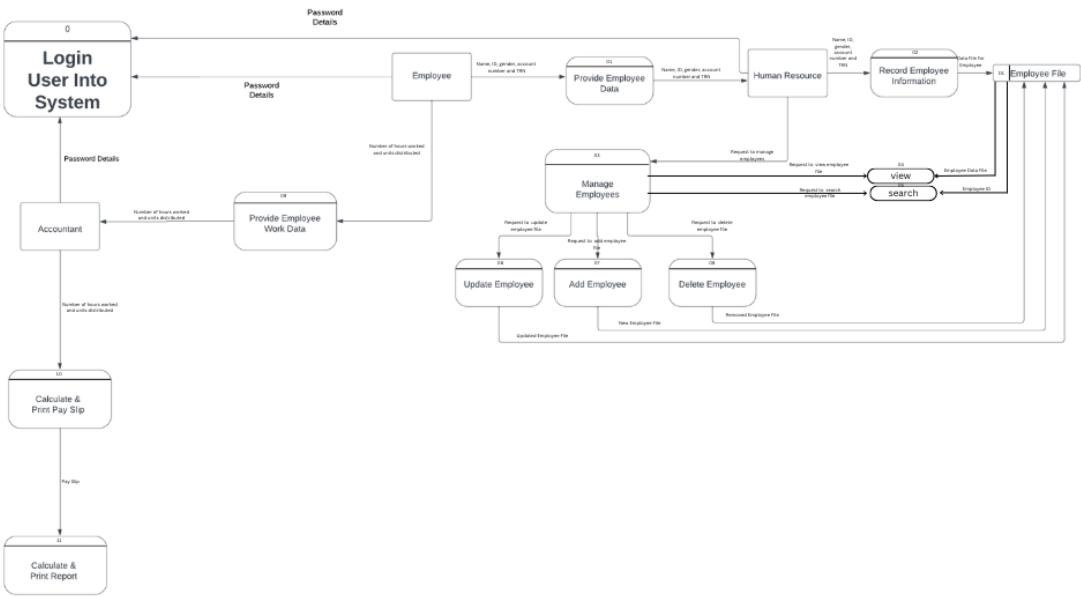
hindering program development. The interview covers management's goals for the digital solution, employees' perspectives on challenges during the transition, and IT specialists' technical insights into existing systems and possibilities for integration. With that, only conducting interviews with key people of these departments, to collect the most meaningful data in an efficient manner. Specific issues detailed from the interview were restrictive nature in employee management, the inconsistency in employee data in program's system in comparison to file's system and functional features failing to operate successfully/properly.

A questionnaire is a method for collecting information, using a set of questions. In this case, an electronic format with a semi-structured style, combining close-ended and open-ended questions, was employed. The purpose was to collect structured quantitative data on employee satisfaction, concerns, and opinions about the current payroll system. The benefits of using a questionnaire include obtaining quantifiable data for statistical analysis and capturing a broad perspective from the entire workforce. The questionnaire was distributed to all employees to identify their satisfaction, concerns, and opinions, seeking feedback for potential improvements. As well as being mindful of the respondents' time, the questionnaire was kept succinct and easily interpretable for quick assessment. In response to the questionnaire, it's been sighted that users are pleased with the system but wish to see a potential improvement in user friendliness, but more importantly functionality as the company begins to grow. Apart from this, ideas were suggested to optimize the current system to meet the new requirements of the firm.

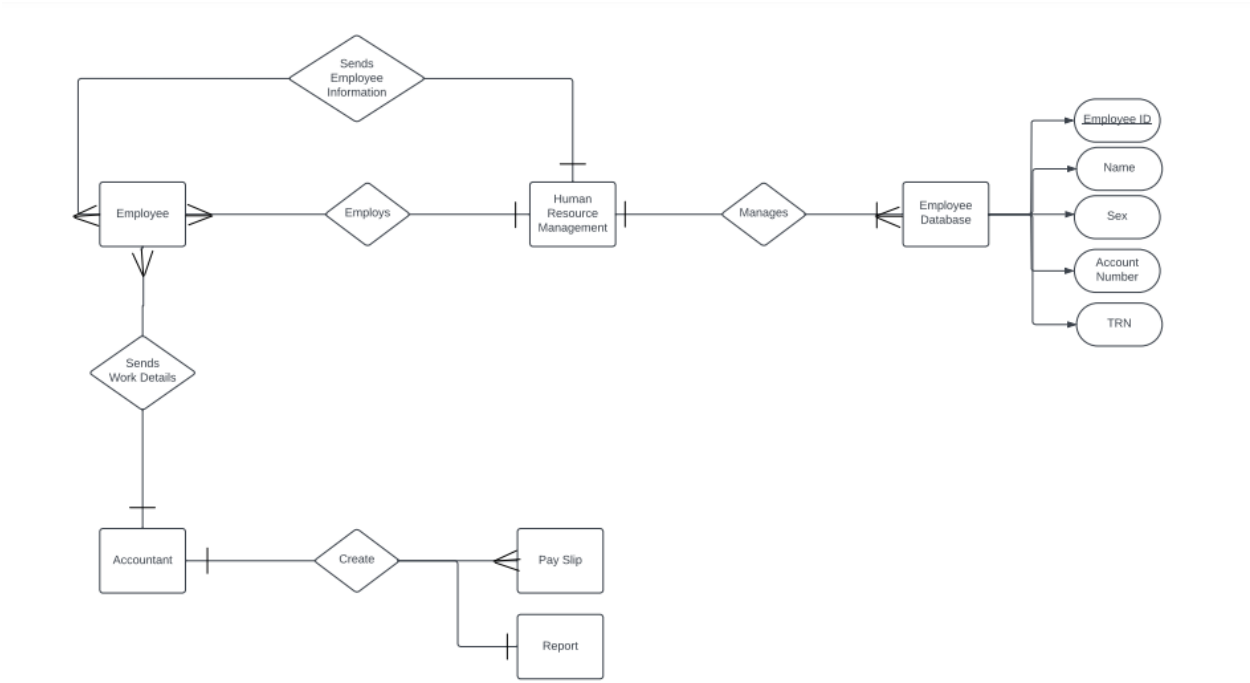
DFD: Context Level



DFD: Level 1



Entity Relationship Diagram



<u>Field Name</u>	<u>Description</u>	<u>Data Size</u>	<u>Data Type</u>	<u>Required</u>
<u>Employee ID</u>	<u>Unique attribute number to identify employee within the firm.</u>		<u>Number</u>	<u>Yes</u>
<u>Name</u>	<u>The first and last name of the employee.</u>		<u>Text</u>	<u>Yes</u>
<u>Sex</u>	<u>The specific gender (male/female) of the employee.</u>		<u>Character</u>	<u>Yes</u>
<u>Account Number</u>	<u>The number that identifies the account number of the employee's bank details,</u>		<u>Number</u>	<u>Yes</u>
<u>TRN</u>	<u>Unique attribute</u>		<u>Number</u>	<u>Yes</u>

	<u>number given upon filing of tax registration number for employee.</u>			
--	--	--	--	--

Functional Requirements

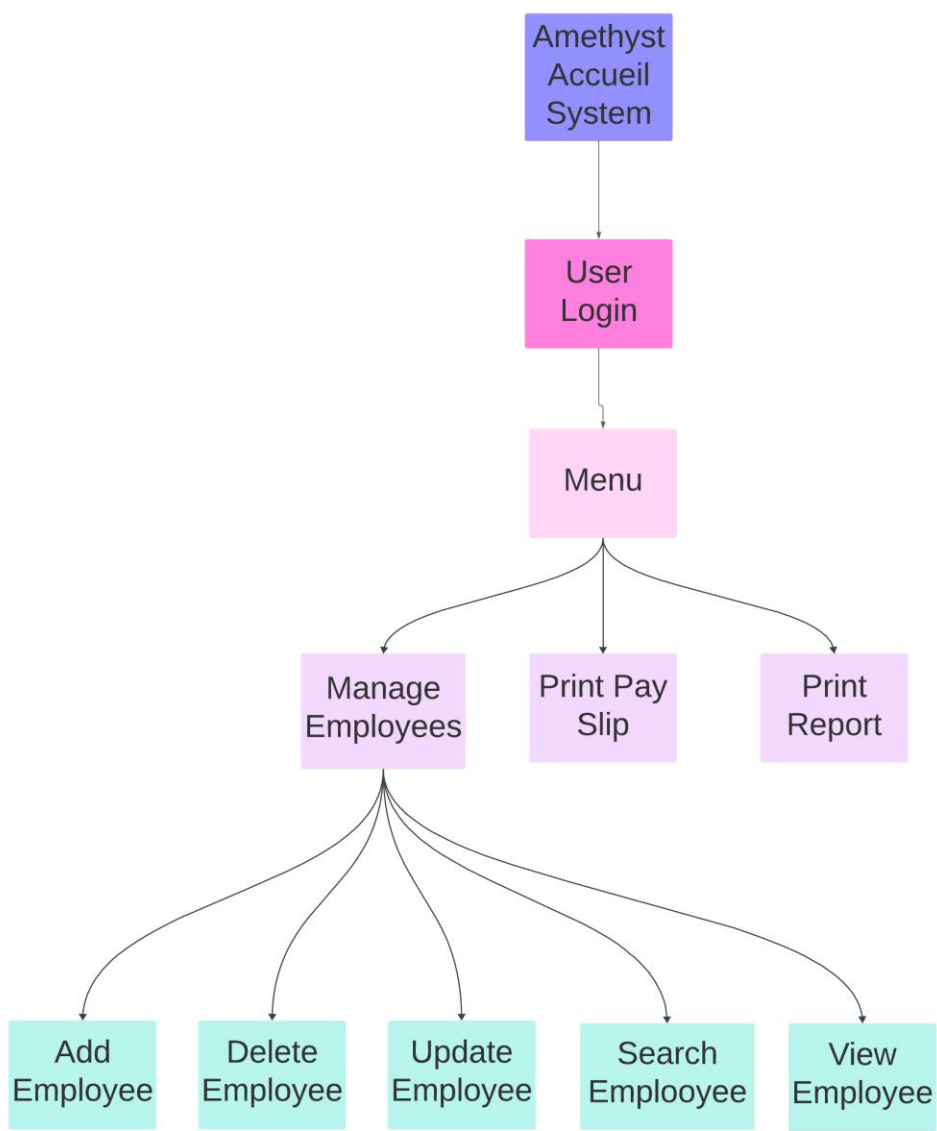
1. The system will be able to store the net pay value of the employees in a database
2. The system will be able to store the pay slips of every employee in a database
3. The system will be able to store the company's report in a database
4. The system will be able to properly delete employee records
5. The system will be able to add more records to the employee file, by expanding the limit of employees, and display them on the print screen.
6. The system will allow the employee data to be sorted given the users' criteria
7. The system will have an improved search function that allows the employee to be searched by their ID and print all pieces of information associated with that ID/employee.
8. The system will keep track of the date to which employee slips and reports have been filed.

Non-Functional Requirements

1. Login passwords must be inputted using password masking algorithms.
2. The system's code should be written in C Programming Language
3. The system should be available 99.99% of the time, excluding scheduled maintenance of 30 minutes per month.
4. The system should support major web browsers (Chrome, Firefox, Safari) and operating systems (Windows, macOS, Linux)
5. The system's usability should be of ease in terms of user-friendliness, with readable and content attractive layouts to invite and assure a smooth process for the users.
6. The system will be able to handle a great amount of information and values by using a series of functions that will clear the screen, as well as clear the memory after a certain amount of information is inputted into the system.

Design Specifications

Structured Chart



HIPO Chart

1. Main Module

- Input: None
- Process:
 - Display welcome message
 - Prompt for password
 - Call password authentication function
- Output: Access granted or denied

2. Password Authentication Function

- Input: User-entered password
- Process:
 - Compare entered password with the correct password
 - Allow access if correct, else allow three attempts
- Output: Access granted or denied

3. Employee Management Function

- Input: User-selected option
- Process:
 - Display menu options for managing employees
 - Execute selected option:
 - Display employees
 - Update employee records
 - Search for an employee
 - Delete an employee

- Add a new employee
- Interact with employee.txt file for data manipulation
- Output: Display employee records or perform specified action

4. Pay Slip Generation Function

- Input: Employee data (hours worked, commissions, deductions)
- Process:
 - Calculate components of pay slip (basic pay, overtime pay, etc.)
 - Display pay slip on screen
 - Append pay slip to payslip.txt file
- Output: Display pay slip

5. Report Generation Function

- Input: Total net pays, commissions, deductions
- Process:
 - Prompt user to print report
 - If selected, display report summary
 - Append report to report.txt file
- Output: Display report summary or prompt

6. File Handling

- Input: Data from text files (employee.txt, payslip.txt, report.txt)
- Process:
 - Read employee records for display, updating, searching, or deletion
 - Write employee data, pay slips, or reports to corresponding files

- Output: Manipulation of text files

7. Other Functions

- Input: None
- Process:
 - Clear screen
 - Set text color
 - Set background color
- Output: Screen display formatting

8. Main Function

- Input: None
- Process:
 - Call welcome function
 - Call password function
 - Call employee management, pay slip generation, and report generation functions
- Output: Program execution

User Interface

The Amethyst Accueil System boasts a command-line-like interface, optimizing response times across its array of functions. This interface eliminates the need for employee users to memorize commands for system navigation. Instead, it offers graphical menus displaying user options, all meticulously crafted with code.



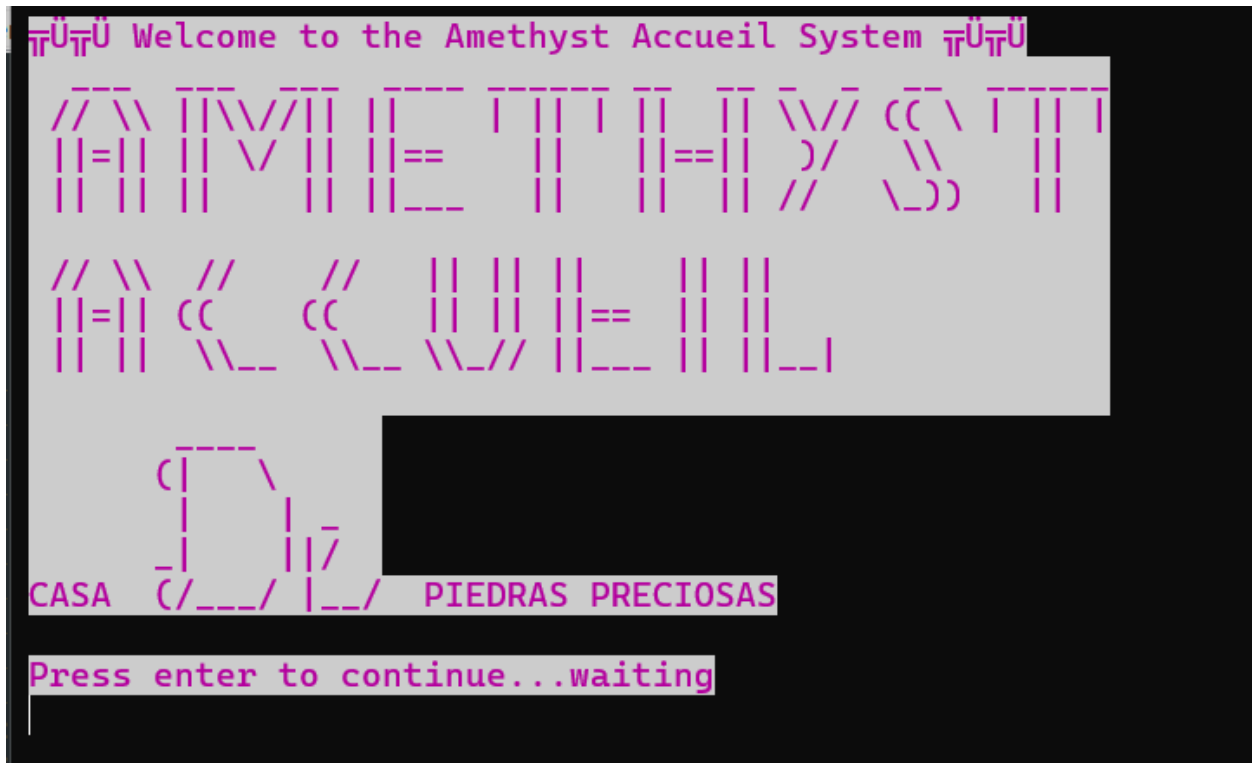


Figure 1. The first screen output to the user is the welcome interface, which displays an inviting greeting to the user, “Welcome to the Amethyst Accueil System”, whilst also prompting the user to login into the system to then be granted access to the system. The name of the system was enlarged in perspective to the other wording to draw emphasis to the firm’s name, whilst also utilizing mostly simplistic font for its readability. The color white was selected to make stark contrast amidst the pink background.

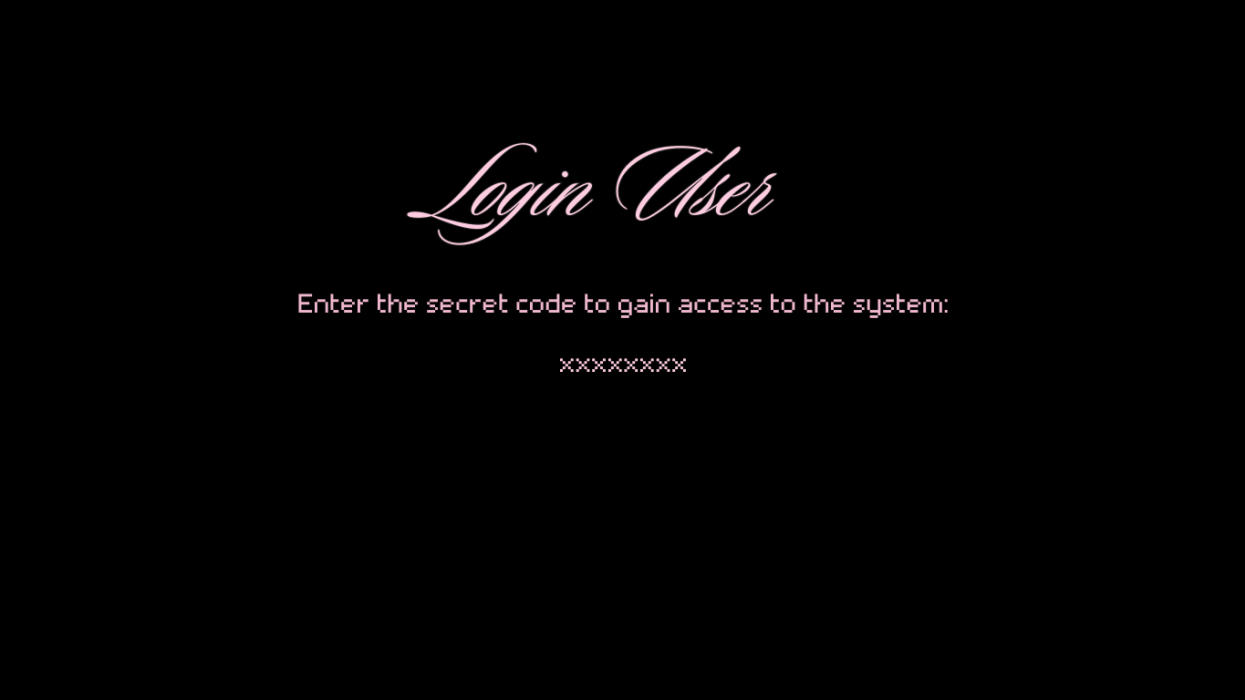


Figure 2. The second screen represents the login screen of the system, this is wherein users are given the ability to input the password to enter the system. After three attempts the user will be logged out of the system and be faced with program termination. In comparison to the welcome screen, the login screen takes on a darker layout to represent the locked state of the system.

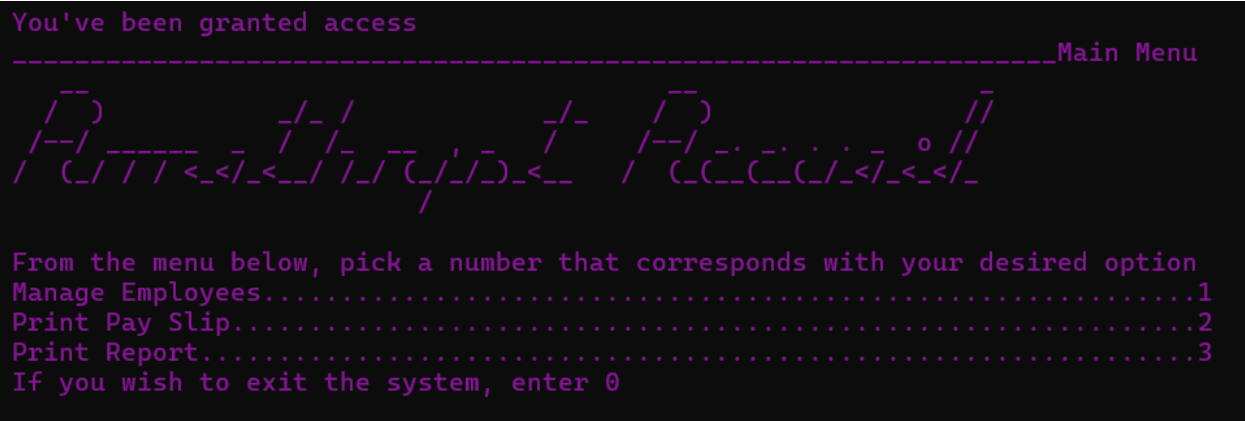
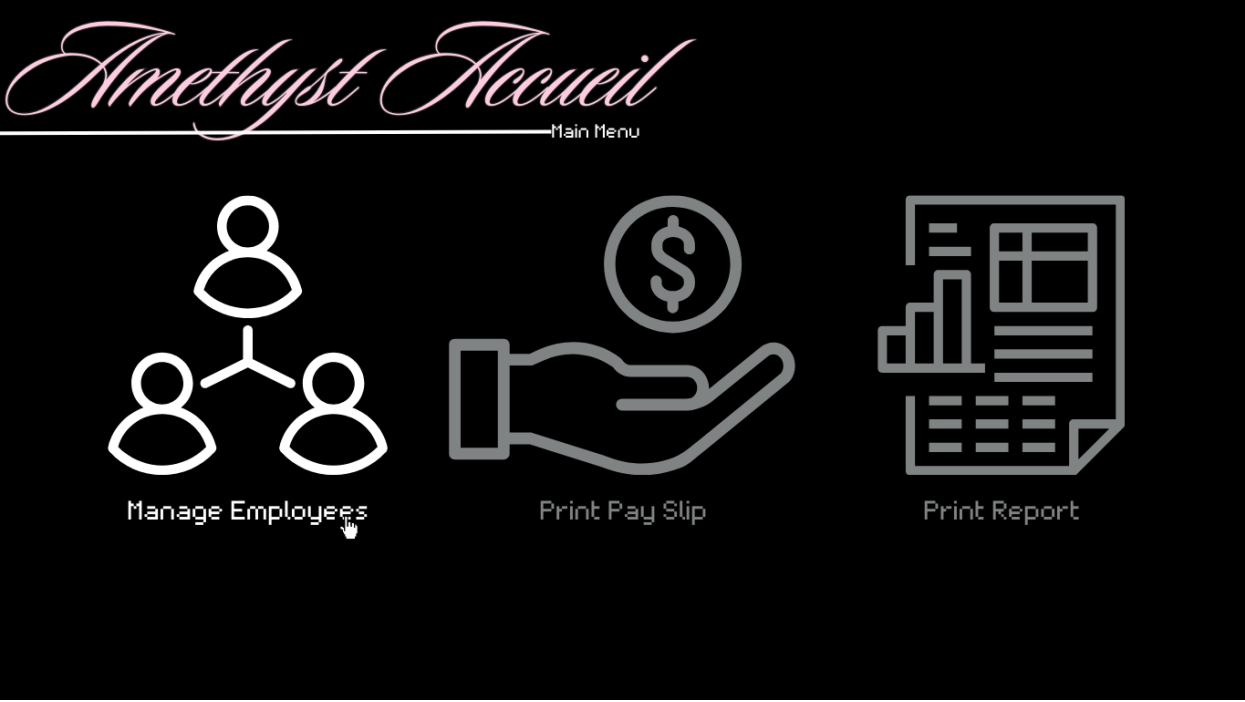


Figure 3. This screen represents the main menu of the system, providing users with the system functions of managing the employees of the database, printing payslip (s) and/or printing report (s). All icons that represent these functions are colorized as light grey, but upon the hover of the mouse over the icon, will turn white to represent the potential selection of users.

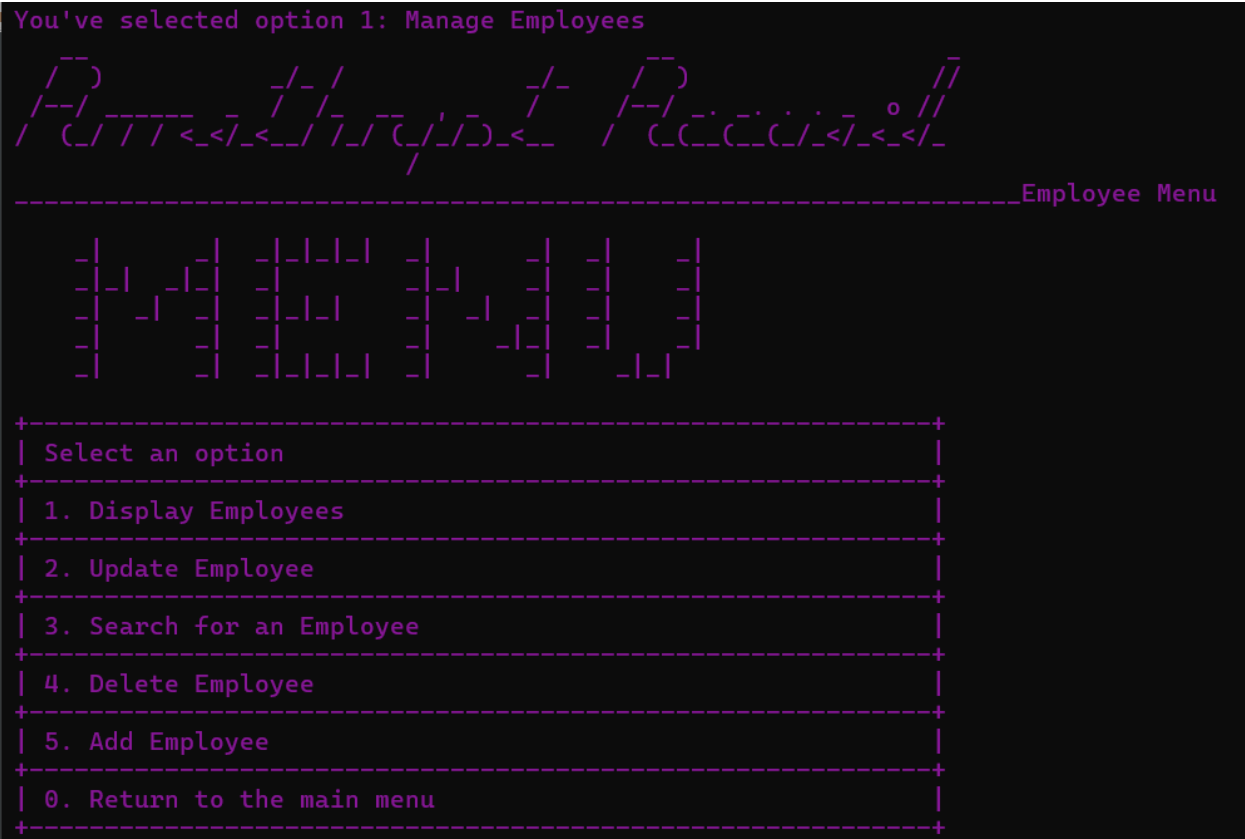
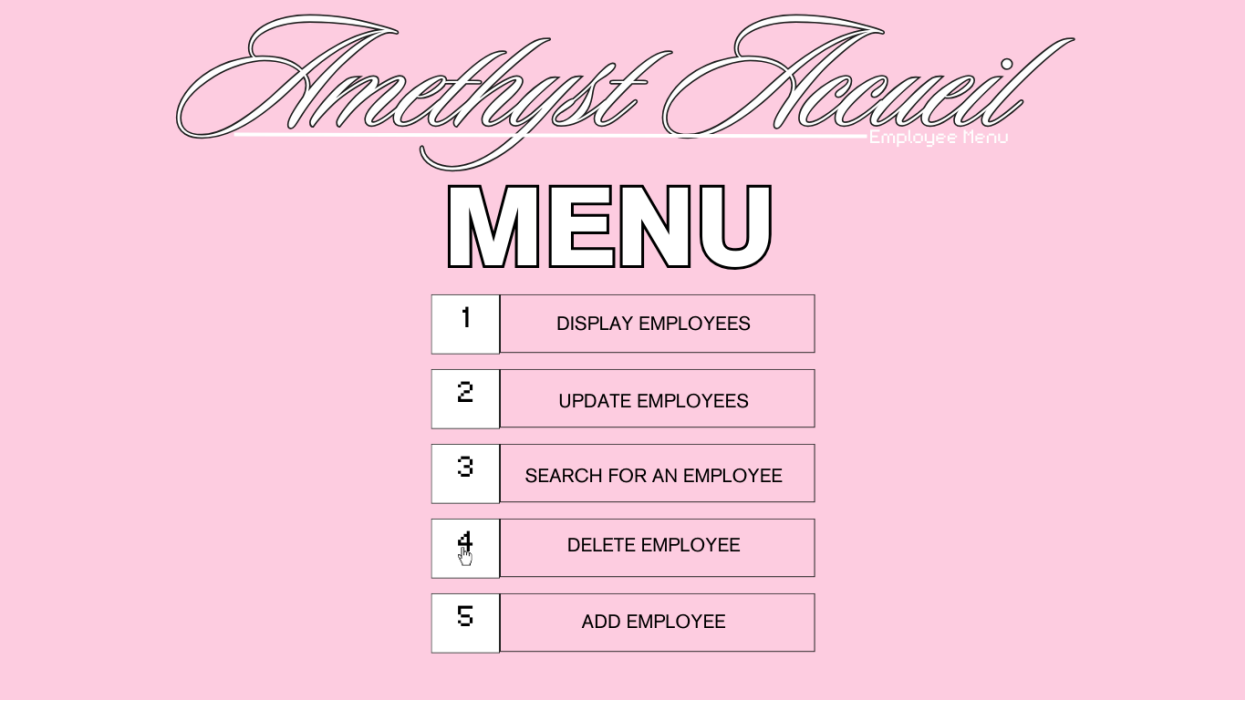


Figure 4. Figure four represents the subsequent menu after selecting the system function of “manage employees”. It prompts the user with a list of options as to how they might want to manage the employee database, from adding, deleting,

updating, viewing, or searching for an employee. In the top right corner, lies a link that upon interaction will send the user back to their previous page, in this case the main menu.

Report Design

PAY SLIP PROCESSING...



Amethyst Account - Distribution Firm Ltd.			
PAY SLIP FOR ID: 05			
EARNINGS	HOURS	RATE	CURRENT
STANDARD PAY	40	65.00	2600.00
OVERTIME PAY	0	1.50	00.00
COMMISSION			50.00
GROSS PAY			2650.00
DEDUCTIONS		CURRENT	
SOCIAL SECURITY TAX		26.50	
STATE INCOME TAX		185.50	
NHT (NATIONAL HOUSING TRUST)		160.50	
PENSION TAX		662.50	
INSURANCE TAX		79.50	
TOTAL DEDUCTIONS		1114.50	
NET PAY		1535.50	

PROCEED



=====Amethyst Accueil - Distribution Firm Ltd=====				

-----Pay Slip for ID: 11-----				

Earnings	Hours	Rate	Current	
Standard Pay	40	65.00	2600.00	
Overtime Pay	16	1.50	1560.00	
Commission			200.00	
Gross Pay			4360.00	
=====				
Deductions		Current		
Social Security Tax		43.60		
State Income Tax		305.20		
NHT		174.40		
Pension Tax		1090.00		
Insurance Tax		130.80		
Total Deductions		1744.00		
Net Pay		2616.00		

Press enter to proceed				

Figure 5. Figure 5 illustrates the report page of the system function, “Print Payslip” after employee work data has been processed. It uses a minimalistic white and black layout for ease of legibility and comparison, alongside the usage of simple word font and size. There is a large pink button the left-hand side, that indicates the option to view the next payslip of an employee.

Report Design



Figure 6. Figure 6, similarly, to figure 5 displays the report page of the system function, “Print Report” after cumulative pay slip data has been processed. It also uses a minimalistic white and black layout for ease of legibility and comparison, alongside the usage of simple word font and size. There is a large pink button the right-hand side, that indicates the option to proceed to the next page of the system, which in this case, would be the main menu.

Aradhyest Account
View Employee



EMPLOYEE ID: 01

NAME OF EMPLOYEE:
RAM TITAN

SEX:
M

Employee's Account Number:
54321

Employee's Tax Registration Number:
123456789

NEXT EMPLOYEE

```
Enter the employee ID to search for: 8
Employee found:
ID: 8
First Name: Robyn
Last Name: Sterling
Sex: F
Account Number: 425367
TRN: 132536527
```

Figure 7. Figure 7 represents the search employee screen, which outputs the headers and information of employees for which the user wished to search. Much like previous screens, the font and size in this screen stays consistent in its readability, as such key information should be easily read without difficulty.

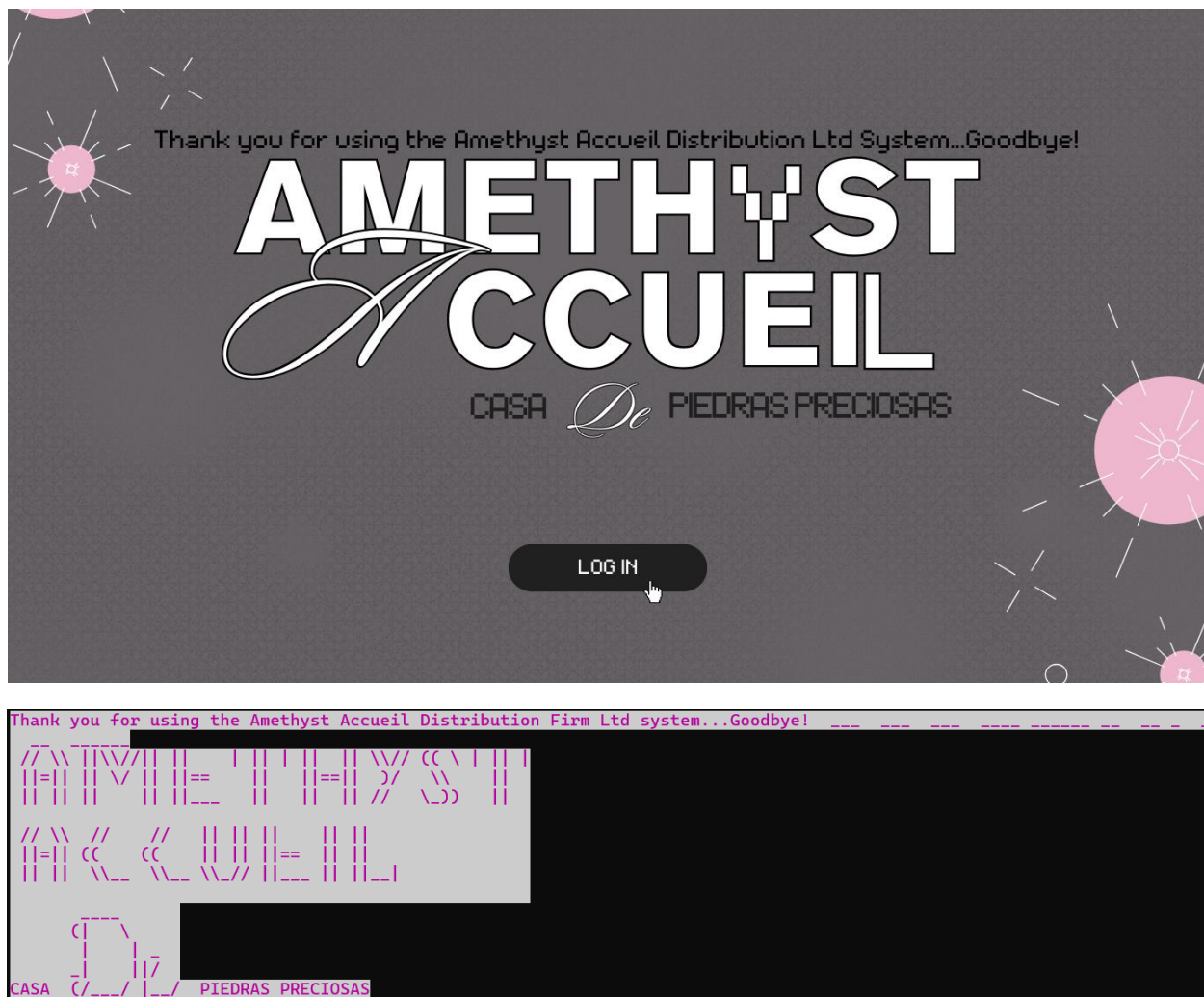


Figure 8. Figure 8 represents the screen presented upon exit of the system, it is the invert of the welcome screen to represent a closed system, using darker colors, but similar word formats to remain consistent.

Technical Manual

Module 1: mor

Narrative

This module will serve to provide the entirety of the code (except for what function “mor_alt_BG” would cover) with graphics. Within this module, contains an ANSI (American National Standards Institute) color code that will print unto the parts of the code, the code being a regular purple color for the text primarily.

Pseudocode:

void mor ()

Start

print (“\33[0;35m = purple”)

End

Module 2: mor_alt_BG

Narrative

This module will serve to provide the welcome and farewell page of the code with graphics. Within the module, two ANSI (American National Standards Institute) color codes will be printed unto the parts of the code, the first being a bold high intensity color of purple that will affect the texts, and the second being regular white color that will affect the background of the texts.

Pseudocode

void mor_alt_BG ()

Start

print (“\e[1 = bold; 95m = high purple”)

print (“\e[47m = white”)

Stop

Module 3: welcome

Narrative:

This module will serve to welcome the user to the system, with a generic welcoming message.

Pseudocode:

```
void welcome ()
```

START

call to the mor_alt_BG function

```
print "°° Welcome to the Amethyst Accueil System °°"
```

Print " _____ "

```
Print "//\\ \\ |\\ \\ \\ /| | | | | | | | \\ \\ \\ /((\\ \\ | | | |"
```

```
Print " ||=|| || \V || ||== || ||==|| )/ \\\ || "
```

```
Print " |||||   |||_  || | || // \_\)) || "
```

Print " "

```
Print "//\\ // // || || || || || "
```

```
Print " || = || (( (( || || || == || || "
```

```
Print " || || \_ \_ \\ \_ / || _ || || _ "
```

Print "

Print " _____ "

Print " (| \\" "

Print " | | _ "

Print " _ | | / "

Print "CASA (__/_ |_/_ PIEDRAS PRECIOSAS"

END

Module 4: goodbye

Narrative:

This module will serve to wish the user off the system, with a generic farewell message.

Pseudocode:

```
void goodbye ()
```

```
START
```

```
    call to the mor_alt_BG function
```

```
    print "Thank you for using the Amethyst Accueil Distribution Firm Ltd system...Goodbye!"
```

```
    Print " _____"
```

```
Print "//\\\\|\\\\W/| | | | | | | |\\\\W/((\\| | | |"
```

```
Print "||=| | |\\| | | |= | | |=| | )/ \\ \\ | | "
```

```
Print "| | | | | | | _ | | | | // \\ ) | | "
```

```
Print " "
```

```
Print "//\\ // // | | | | | | | "
```

```
Print "||=| |( ( | | | | |= | | | "
```

```
Print "| | | \\_ \\_ \\ \\ \\ // | _ | | | _ | "
```

```
Print " "
```

```
Print " _____"
```

```
Print " (| \\ "
```

```
Print " | | _ "
```

```
Print " _| ||/ "
```

```
Print "CASA (\\_/_/_/ PIEDRAS PRECIOSAS"
```

```
END
```

Module 5: WIPE

Narrative:

This module will serve to clear screen after each usage.

Pseudocode:

void WIPE ()

START

 system clear screen

END

Module 6: password (void)**Narrative:**

This module will serve to secure the system with a password. It will declare 2 local variables, “ent_pass” and “count” as integers, and initialize two variables, i to 3 and x to 0. After declaring and initializing, the “mor” function will be called to provide color to the module, after which the user will be prompted to enter a secret pin to gain access to the system, this pin will be stored in the variable previously mentioned, being “ent_pass”. The program will read this entered pin and then immediately after clear the screen, by calling to the “WIPE” function. A conditional structure (nested if) will then declare if the user has gained access or not to the system. If the user had initially entered “5791” as the pin, then the system will output the text “You’ve been granted access” and then call to the “menu” function, else the system will call to the “WIPE” function and tell the user that the secret pin they’ve entered is incorrect and to try again, after which the program will use an iteration structure (for loop) to give the user 3 attempts to gain access. The “count” variable will provide the user with the amount of attempts they have remaining after each attempt is made. If any time within those three attempts the pin is correct, then they’ll be taken to the menu; the “menu” function will be called, else the program will output “Access Status = Not granted” and terminate.

Pseudocode:

password ()

START

ent_pass, count, i, x as integer

 i = 3

x = 0

Print " _ _ _ "

Print " _/ ') / "

Print " / _ _ , o _ _ / / _ _ "

Print "/ _ _ () () _ < / / < _ (_ / / _) _ < / / (_ "

Print " _ / | _ "

Print " _ | / _ "

Call to the "mor" function.

print "Enter the secret code to gain access to the system"

read ent_pass

call to WIPE function

if (ent_pass = 5791) then

print "You've been granted access"

call to menu function

endif

else

system clear

print "Incorrect Secret Code...Try Again!"

print "You have three attempts"

for x=1 to 3 DO

print "Enter the secret code"

read ent_pass

system clear

x = x + 1

```

        if (ent_pass == 5791) then
            print "You've been granted access"
            calls to the menu function
        end if

        if (ent_pass <> 5791) then
            print "Remaining Attempts:",3-x
            print "Access Status = Not Granted"
        endif
    endfor

endif

END

```

Module 7: menu

Narrative

This module will serve to present to the user the main menu of the system. It will declare two variables as integers, x and opt, while initializing x to 0. After which, a do...while loop will loop the menu until the user chooses to terminate it with the terminator 0. Within the loop will present the user with four options to choose from; manage employees (represented by the number 1), print the pay slip (represented by the number 2), print the report (represented by the number 3), or exit the system (represented by the number 4). Subsequently, a switch structure will be used to deliver the tasks of each option, if they choose 1, then the "employf" function will be called, if they choose 2, then the "PAYSLIP" function will be called, if they choose 3, then the "report" function will be called and if they choose to exit the system, then the "goodbye" function will be called. If the user happens to enter any value that doesn't represent any of the options displayed, the system will print "Invalid Choice" and the menu will reappear.

Pseudocode:

```

menu()

START

x, opt as integer

```

```

x = 0
do while (opt > 1 OR opt < 5)

```

```

    Print "_____Main
Menu"
Print "  _ _ _ "
Print " / )  _/_  _/_ / )  // "
Print "/--/ _____ _ / /_ _ , _ /  /--/ _ . _ . . _ o // "
Print "/ ( / / / <_</<_ / / ( / / ) _<_ / ( ( ( ( / _</<_</<_ "
Print "      /      "

```

```

    print "From the menu below, pick a number that corresponds with your desired option"
    print "Manage Employees.....1"
    print "Print Pay Slip.....2"
    print "Print Report.....3"
    print "If you wish to exit the system, enter 0"
    read opt
    call to WIPE function

```

```

case based on opt

case = 1:

    print "You've selected option 1: Manage Employees"
    call to "employ" function


case = 2:

    print "You've selected option 2: Print Pay Slip"
    call to "PAYSLIP" function

```

```

case =3:
    print "You've selected option 3: Print Report"
    call to "report" function

case = 0: call to "goodbye" function
    exit (0)

default
    print "Invalid Choice"

ENDCASE

```

```

endwhile
return opt
END

```

Module 8: getCurrentDate

Narrative

This module functions to retrieve the current date, utilizing the `getCurrentDate` function. It initiates by acquiring the current time and storing it in variable `t`. Subsequently, it employs the `localtime` function to convert the time value into a structure (`tm`) containing information about the local time. Then, utilizing the `strftime` function, it formats this time information into a string with the desired format "YYYY-MM-DD" and stores it in the `date` variable passed as an argument.

Pseudocode:

Declaring void getCurrentDate(char *date)

START

```

time_t t = time (NULL)
struct tm * tm = localtime (&t);
strftime (date, 20, "%Y-%m-%d", tm);

```

STOP

Module 9: PAYSALIP

Narrative

This module aims to generate and print pay slips for each employee within the system. It begins by declaring various variables, including integers like ``i``, ``hrs_wrk``, ``ov_hrs``, and ``units_dis``, along with real numbers such as ``bas_pay``, ``g_p``, ``ot``, ``com``, ``net_p``, ``ss_tax``, ``st_inc_tax``, ``NHT``, ``pen``, ``insur``, and ``t_deduc``. Additionally, a string variable named ``cho`` is declared.

The constants ``bas_hrly_r``, ``over_r``, and ``bas_hrs`` are initialized, representing hourly rates, overtime rates, and standard hours respectively.

The process iterates through each employee, prompting for hours worked and units distributed. Based on these inputs, it calculates overtime pay, basic pay, commission, gross pay, and various deductions including taxes, pension, and insurance. The net pay is then determined.

The user is given the option to print the pay slip for each employee. If chosen, the program generates a formatted pay slip and saves it to a file named "payslip.txt". If not, it proceeds to the next employee. If an invalid response is entered, it informs the user and continues. After processing all employees, it returns to the main menu function.

The program utilizes the ``WIPE`` function to clear the screen and pauses for user acknowledgment before proceeding to the menu.

Pseudocode:

Declaring void PAYSALIP (float *total_np, float *total_com, float *total_td)

START

char date [20]

call to getCurrentDate (date)

```

hrs_work, i, ov_hrs, units_dis as integer
bas_pay, g_p, ot, com, net_p as real
ss_tax, st_inc_tax, NHT, pen, insur, t_deduc as real
cho as string
bas_hrly_r, over_r as float
bas_hrs as integer
const bas_hrly_r = 65.00
const over_r = 1.5
const bas_hrs = 40

print "For which employee would you like to print the payslip?"
read empid

file named "employee.txt" open in read-only mode
    if (file == NULL) then
        print "An error occurred while opening this file."
        return;
    endif

while reading from file until all data is processed{
    IF (EMP [t_emp].id = empid) Then

        print "Processing for Employee ID", emp_id[i]
        print "Enter the number of hours worked by the employee"
        read hrs_wrk

        total_hw = total_hw + hrs_wrk

        if (hrs_wrk > bas_hrs) then

```

```

    ov_hrs = hrs_wrk – bas_hrs
    ot = 00.00
    bas_pay = bas_hrly_r * bas_hrs
endif

    else if (hrs_wrk < bas_hrs) then
    ov_hrs = 0
    ot = 00.00
    bas_pay = bas_hrly_r * bas_hrs
    endif

print “Enter the number of units distributed by the employee”
read units_dis
total_ud = total_ud + units_dis

    if (units_dis >= 200 AND units_dis < 250) then
        com = 50.00
        total_com = total_com + com
    endif

    else if (units_dis >= 250 AND units_dis < 300) then
        com = 100.00
        total_com = total_com + com
    endif

    else if (units_dis >= 300 AND units_dis<400) then
        com = 150.00
        total_com = total_com + com
    endif

    elseif (units_dis >= 400) then
        com = 200.00
        total_com = total_com + com

```

```

endif

else

com = 00.00

total_com = total_com + com

endif

g_p = com + ot + bas_pay

ss_tax = g_p * 0.01
st_inc_tax = g_p * 0.07
NHT = g_p * 0.04
pen = g_p * 0.25
insur = g_p * 0.03
t_deduc = ss_tax + st_inc_tax + NHT + pen + insur
total_td = total_td + t_deduc

net_p = g_p - t_deduc
total_np = total_np + net_P
system clear screen

print "Would you like to print the employee's pay slip , Yes or No?"
read cho

Call to the WIPE function

if (cho = "Yes")then

Print          "Amethyst Accueil - Distribution Firm Ltd"
Print "-----\n"
Print          "Pay Slip"

```

```

Print "-----\n"
Print "Earnings      |   Hours      |   Rate      |   Current"
Print "Standard Pay:", |   bas_hrs    |   bas_hrly_r |   bas_pay
Print "Overtime Pay:", |   ov_hrs     |   over_r     |   ot
Print "Commission value:", |           |           |   com
Print "Gross Pay:",    |           |           |   g_p
Print "-----"

Print "Deductions      |           |   Current"
Print "Social Security Tax |           |   ",ss_tax
Print "State Income Tax   |           |   ",st_inc_tax
Print "NHT                |           |   ",NHT
Print "Pension Tax        |           |   ",pen
Print "Insurance Tax      |           |   ",insur
Print "Total Deductions   |           |   ",t_deduc
Print "Net Pay            |           |   ",net_p

```

file3 named "payslip.txt" open in append mode

if (file3 == NULL) then

 print "An error has occurred while opening this file"

 return 1

endif

writefile(file3,

"|=====

=Date: %s", date)

writefile(file3, "|=====Amethyst Accueil - Distribution Firm

Ltd=====")

writefile(file3,

"|_____

_____")

```

writefile(file3, "| _____ Pay Slip for ID:
%d _____")

writefile(file3,
"| _____
_____")

writefile(file3, "| Earnings      |      Hours      |      Rate      |      Current
")

writefile(file3, "| Standard Pay  |      %d      |      %.2f      |      %.2f
", bas_hrs, bas_hrly_r, bas_pay)

writefile(file3, "| Standard Pay  |      %d      |      %.2f      |      %.2f
", hrs_wrk, bas_hrly_r, bas_pay)

writefile(file3, "| Overtime Pay |      %d      |      %.2f      |      %.2f
", ov_hrs, over_r, ot)

writefile(file3, "| Commission  |      |      |      %.2f      ", com)
writefile(file3, "| Gross Pay   |      |      |      %.2f      ")

writefile(file3,
"|=====
=====")

writefile(file3, "| Deductions      |      Current      ")

writefile(file3, "| Social Security Tax      |      %.2f      ", ss_tax)
writefile(file3, "| State Income Tax      |      %.2f      ", st_inc_tax)
writefile(file3, "| NHT      |      %.2f      ", NHT)
writefile(file3, "| Pension Tax      |      %.2f      ", pen)
writefile(file3, "| Insurance Tax      |      %.2f      ", insur)
writefile(file3, "| Total Deductions      |      %.2f      ", t_deduc)
writefile(file3, "| Net Pay      |      %.2f      ", net_p)

writefile(file3,
"| _____
_____")

fclose(file3)

print "Press enter to proceed"

```

```

char c = getchar()
ELSE IF strcmp(cho, "No") == 0 THEN
    PRINT "To proceed, press enter...\n"
    ELSE
        PRINT "Invalid Response...Proceeding...Press enter\n"
    END IF

    getch()
    END IF
END WHILE

system("pause")
WIPE ()
PRINT "\nPress the enter key to go back to the menu\n"
CHAR c = getchar()
END FUNCTION

```

Module 10: report

Narrative:

Narrative:

This module is designed to generate and print a report summarizing various key metrics related to employee performance and financials. It begins by invoking the `getCurrentDate` function to retrieve the current date.

The `WIPE` function is called to clear the screen for better readability. The user is prompted to decide whether they want to print the report. If the response is "Yes," the program proceeds to generate a formatted report containing the total units distributed, total hours worked by employees, total net pays distributed, total commissions awarded, and total deductions subtracted from employees' gross pays.

The program then pauses for user acknowledgment before proceeding to write the report to a file named "report.txt." If any errors occur during file handling, an error message is displayed.

After writing the report to the file, the program pauses again for user acknowledgment before calling the `WIPE` function to clear the screen once more.

If the user chooses not to print the report, the program simply proceeds to the end without generating the report or writing it to a file.

Pseudocode:

Declaring void report (float total_np, float total_com, float total_td)

START

ans as string

ans as date

Call to the getCurrent Date (date) function

call to the "WIPE" function

print "Would you like to print the Report, Yes or No? "

read ans

if (ans = "Yes") then

print "+++++Amethyst Accueil Distributing Firm Ltd.+++++"

print"+++++"

T+++++
printf ("+++++R E P O R

print "The total amount of units distributed is

", total_ud

print ("The total amount of hours worked by employees is:

", total_hw

printf ("The sum of net pays distributed to employees is: ",

total_np

```

                                printf ("The sum of commissions awarded to employees is:
                                ", total_com
printf ("The sum of deductions subtracted from Employee's Gross Pays is:                                ", total_td
                                system pause

                                // Creating and writing to file4
                                file4 named "report.txt" open in write mode
                                if (file4 == NULL) then
print "An error has occurred while opening file4"
return
endif

writefile(file4, "Date: %s\n", date)
writefile(file4, "Total Units Distributed: %d\n", total_ud)
writefile(file4, "Total Hours Worked: %d\n", total_hw)
writefile(file4, "Total Net Pays Distributed: %.2f\n", total_np)
writefile(file4, "Total Commissions Awarded: %.2f\n", total_com)
writefile(file4, "Total Deductions: %.2f\n", total_td)
fclose(file4)

system("pause")
endif

WIPE() // calls to the wipe function
}

                                ENDIF

END

```

Module 11: employf

Narrative

This module facilitates the management of employee records within a system. It begins by initializing various variables and pointers needed for the operation. The user is presented with a menu offering several options, including displaying employees, updating employee information, searching for an employee, deleting an employee, adding a new employee, or returning to the main menu.

Upon selecting an option, the program executes the corresponding functionality:

1. Display Employee: The system opens the "employee.txt" file and reads and displays each line, presenting the employee records to the user.
2. Update Employee: The system prompts the user to enter the ID of the employee they wish to update. It then searches for the employee in the file, allows the user to update the employee's details, and writes the updated information back to the file.
3. Search for an Employee: The user is prompted to enter the ID of the employee they want to search for. The system reads through the file, searching for the specified ID, and if found, displays the employee's details.
4. Delete Employee: The user is prompted to enter the ID of the employee they want to delete. The system reads through the file, deletes the specified employee's record, and updates the file accordingly.
5. Add Employee: The system prompts the user to enter details for a new employee, including ID, first name, last name, sex, account number, and TRN. It then appends this information to the "employee.txt" file.

The module operates in a loop until the user chooses to return to the main menu by entering 0. If the user selects an invalid option, the system prompts them with an "Invalid Choice" message. Overall, this module provides essential functionality for managing employee records efficiently.

Pseudocode:

Declaring employf (int *id, char*fname, char *lname, char *sex, int *acc, int *trn)

START

FUNCTION flushInputBuffer()

// Flush the input buffer

READ character FROM stdin UNTIL character = '\n' OR character = EOF

END FUNCTION

DECLARE

employees AS employ POINTER

numEmployees, arraySize, i, sel, search_num AS INTEGER

foundb, found AS BOOLEAN

c, ano AS CHARACTER

line[MAX_LINE_SIZE] AS STRING

arraySize = t_emp

foundb = false

Repeat until sel equals 0:

Print " _____ "

Print " /) _ _ / _ _ /) _ _ // "

Print " /--/ _____ _ / / _ _ , _ / /--/ _ _ . . . _ o // "

Print " / (/ / / < _ < / _ < _ / / / (/ /) _ < _ / ((((/ _ < / _ < _ < / _ "

Print " _ _ / _ _ "

Print " _____ Employee
Menu"

Print "\n _ | _ | _ | _ | _ | _ | _ | _ | "

```

Print "  _|_| _|_| _|  _|_| _| _| _|"
Print "  _| _| _| _|_|_| _| _| _| _| _|"
Print "  _|  _| _|  _|  _|_| _| _|"
Print "  _|  _| _|_|_|_| _|  _|  _|_| "
```

```

Print "\n+-----+"
Print "| Select an option                               |"
Print "+-----+"
Print "| 1. Display Employees                               |"
Print "+-----+"
Print "| 2. Update Employee                               |"
Print "+-----+"
Print "| 3. Search for an Employee                         |"
Print "+-----+"
Print "| 4. Delete Employee                               |"
Print "+-----+"
Print "| 5. Add Employee                                   |"
Print "+-----+"
Print "| 0. Return to the main menu                         |"
Print "+-----+"

```

```

    read sel // Scan the user's selection

```

```

WIPE() // Call the WIPE function to clear the input buffer

```

```

    case 1:

```

```

        print("Selected Option: 1...Display Employees");

```

```

    Open file named "employee.txt" in read mode

```

```
IF file == NULL THEN
    PRINT "An error has occurred while opening the file"
    RETURN
END IF

print("Employee records:");

// Read and display each line in the file
Repeat until end of file {
    Read a line from file
    Print the line
}

Close file
break;

case 2:
    print("Selected Option: 2...Update Employee");

    Open file named "employee.txt" in read mode
IF file == NULL THEN
    PRINT "Error opening file."
    RETURN
END IF

    Open file named "copy.employee.txt" in write mode
IF file2 == NULL THEN
    PRINT "Error opening file."
    CLOSE file
    RETURN
```

END IF

print("Enter the ID of the Employee to update:");

read search_num;

// Loop through each record in the original file

Repeat until end of file {

 Read record from file

IF EMP[t_emp].id == search_num THEN

 PRINT "Employee found:"

 PRINT "ID: ", EMP[t_emp].id

 PRINT "First Name: ", EMP[t_emp].fname

 PRINT "Last Name: ", EMP[t_emp].lname

 PRINT "Sex: ", EMP[t_emp].sex

 PRINT "Account Number: ", EMP[t_emp].acc

 PRINT "TRN: ", EMP[t_emp].trn

 PRINT "Enter new first name of Employee:"

 READ EMP[t_emp].fname FROM stdin

 PRINT "Enter new last name for Employee:"

 READ EMP[t_emp].lname FROM stdin

 PRINT "Enter new sex of Employee:"

 READ EMP[t_emp].sex FROM stdin

 PRINT "Enter new Employee's Account Number:"

 READ EMP[t_emp].acc FROM stdin

 PRINT "Enter New Employee's Tax Registration Number:"

 READ EMP[t_emp].trn FROM stdin

found = true

END IF

WRITE line TO file2

END WHILE

CLOSE file

CLOSE file2

IF found THEN

REMOVE("employee.txt")

RENAME("copy.employee.txt", "employee.txt")

PRINT "Record updated successfully."

ELSE

PRINT "Record doesn't seem to exist."

REMOVE("copy.employee.txt")

END IF

BREAK

case 3:

print("Selected Option: 3...Search for an Employee");

Open file named "employee.txt" in read mode

PRINT "Enter the employee ID to search for: "

READ search_num FROM stdin

CLEAR INPUT BUFFER

```
IF file == NULL THEN

    PRINT "An error has occurred while opening the file"

    RETURN

END IF

// Read and search each line in the file

Repeat until end of file {

    Read record from file

        if (EMP[t_emp].id == search_num) {

            // If employee found, print details

            print("Employee found:");

            PRINT "ID: ", EMP[t_emp].id

            PRINT "First Name: ", EMP[t_emp].fname

            PRINT "Last Name: ", EMP[t_emp].lname

            PRINT "Sex: ", EMP[t_emp].sex

            PRINT "Account Number: ", EMP[t_emp].acc

            PRINT "TRN: ", EMP[t_emp].trn

            foundb = true;

            BREAK

        }

    END IF

END WHILE

IF NOT foundb THEN

    PRINT "Employee with ID ", search_num, " not found"

END IF

CLOSE file

BREAK
```

case 4:

PRINT "Selected Option 4...Delete Employee"

Open file named "employee.txt" in read mode

IF file == NULL THEN

 PRINT "An error has occurred while opening the file"

 RETURN

END IF

Open file named "copy.employee.txt" in write mode

IF file2 == NULL THEN

 PRINT "An error has occurred while opening the file"

 RETURN

END IF

print("Enter the ID of the Employee to delete:");

read search_num;

// Loop through each record in the original file

Repeat until end of file {

 Read record from file

 if (EMP[t_emp].id == search_num) {

 // If the ID matches the search ID, skip writing this record to the temporary file

 print("Employee found and deleted:")

 PRINT "ID: ", EMP[t_emp].id

 PRINT "First Name: ", EMP[t_emp].fname

 PRINT "Last Name: ", EMP[t_emp].lname

 PRINT "Sex: ", EMP[t_emp].sex

```

PRINT "Account Number: ", EMP[t_emp].acc
PRINT "TRN: ", EMP[t_emp].trn

foundb = true; // Set found flag to true
    else
        Write record to file2
End If
End While
Close file; // Close the original file
Close file2; // Close the temporary file

if (foundb) {
    // Replace the original file with the temporary file
If foundb Then
    Delete "employee.txt"
    Rename "copy.employee.txt" to "employee.txt"
    Print "Record deleted successfully."
Else
    Print "Record doesn't seem to exist."
    Delete "copy.employee.txt"
End If
BREAK

```

```

case 5:
    print("Selected Option: Add Employee")

    // Open file for appending
    file named "employee.txt" open in append mode

    if (file is NULL):

```

```
print("An error has occurred while opening this file")
```

```
return 1
```

```
endif
```

```
For i from 0 to t_emp
```

```
Print "Enter the employee's ID:"
```

```
Read EMP[i].id
```

```
Clear input buffer
```

```
Print "Enter the employee's first name:"
```

```
Read EMP[i].fname
```

```
Clear input buffer
```

```
Print "Enter the employee's last name:"
```

```
Read EMP[i].lname
```

```
Clear input buffer
```

```
Print "Enter the employee's sex:"
```

```
Read EMP[i].sex
```

```
Clear input buffer
```

```
Print "Enter the employee's account number:"
```

```
Read EMP[i].acc
```

```
Clear input buffer
```

```
Print "Enter the employee's TRN:"
```

```
Read EMP[i].trn
```

```
Clear input buffer
```

Write to file: "%d,%s,%s,%c,%d,%d\n",

EMP[i].id,

EMP[i].fname,

EMP[i].lname,

EMP[i].sex,

EMP[i].acc,

EMP[i].trn

Print "Employee record successfully stored"

// Ask if the user wants to add another employee

IF i < t_emp - 1 THEN

Declare choice as char

Print "Do you want to add another employee? (y/n):"

Read choice

If choice is not 'y' and choice is not 'Y' Then

BREAK

Exit loop

End If

End If

End For

Close file

BREAK

case = 0: call to "menu" function

BREAK

DEFAULT: print "Invalid Choice"

BREAK

END SWITCH

END REPEAT

END

Module 12: main

Narrative

This module serves as the driver of the program.

The program begins by executing a series of functions to initialize the system and welcome the user. First, the ``welcome()`` function is called to greet the user and introduce them to the system. After displaying the welcome message, the program prompts the user to press enter to continue.

Once the user confirms, the ``WIPE()`` function is called to clear the screen, ensuring a clean interface for the next step. Subsequently, the ``mor()`` function is executed, presenting additional information or instructions to the user, possibly related to the system's purpose or functionality.

Following that, the ``password()`` function is invoked to prompt the user to enter a password or authentication information to access the system's main features or functionalities. This step enhances security and ensures authorized access.

After entering the password, the program again calls the ``WIPE()`` function to clear the screen, maintaining a neat display. At this point, the system is ready for further interaction with the user, who can then proceed to access the main menu or perform other tasks as needed.

Overall, this sequence of function calls sets up the initial environment of the system, greets the user, provides necessary instructions, handles authentication, and prepares the interface for user interaction.

Pseudocode:

```
// Function prototype Declarations

void welcome();

void mor();

void password();

void WIPE();

void menu();

void employf();

void report(float total_np, float total_com, float total_td);

void mor_alt_BG();

void goodbye();

void PAYSLIP(float *total_np, float *total_com, float *total_td);
```

```
START
```

```
int main():
```

```
    // Call to the welcome function
    welcome()
```

```
    // Prompt to continue
    print("Press enter to continue...waiting")
    char c = getchar()
```

```
    // Call to the WIPE function
    WIPE()
```

```
    // Call to the mor function
    mor()
```

```
    // Call to the password function
```

```
password()
```

```
// Call to the WIPE function
```

```
WIPE()
```

```
STOP
```

Algorithm Design

Programmer: Robyn Sterling of 602-B

Course: Computer Science

Format: Database and Accounting Styling

Description:

- Manage a set of employees and records
- Features include searching, display, adding, removing, and updating
- Ability to compute individual pay slips and company reports with minimal data input
- Data recorded in separate files for organization

Start Date: November 24, 2022

End Date: April 7, 2024

// Libraries

Include "stdlib"

Include "stdio"

Include "conio"

Include "string"

Include "stdbool"

Include "time"

// Constants

const t_emp = 6

```
const MAX_LINE_SIZE = 100
```

```
// Record Definition
```

```
RECORD employ
```

```
    DECLARE
```

```
        id, acc, trn AS Integer
```

```
        fname, lname AS String
```

```
        sex AS Character
```

```
    END DECLARE
```

```
END RECORD
```

```
// Global Variables
```

```
DECLARE
```

```
    total_ud, total_hw AS Integer
```

```
    total_np, total_com, total_td AS Real
```

```
    file, file2, file3, file4 AS File
```

```
    EMP[t_emp] AS employ
```

```
END DECLARE
```

```
// Algorithm/Function to set text color to magenta
```

```
FUNCTION mor ()
```

```
    PRINT "\033[0;35m" // Set text color to magenta
```

```
END FUNCTION
```

```
// Algorithm/Function to set text color to magenta and background color to light magenta
```

```
FUNCTION mor_alt_BG ()
```

```
    PRINT "\e[1;95m" // Set text color to magenta
```

```
    PRINT "\e[47m" // Set background color to light magenta
```



```

Print " //\\ \\ ||\\ \\ / || || | || || || \\ \\ / (( \\ || || "
Print " ||=|| || \\ || ||== || ||==|| )/ \\ \\ || "
Print " || || || || ||__ || || || // \\ ) || "
Print "
"
Print " //\\ // // || || || || || "
Print " ||=|| (( ( || || ||== || || "
Print " || || \\_ \\_ \\ \\ / ||__ || ||_ | "
Print "
"
Print " ____ "
Print " (| \\ "
Print " | | _ "
Print " _| ||/ "
Print "CASA (\\_/_/_/ PIEDRAS PRECIOSAS"

```

END FUNCTION

// Algorithm/Function to clear the screen

FUNCTION WIPE ()

 system ("cls") // Clears the screen

END FUNCTION

// Algorithm/Function to clear the input buffer

FUNCTION clearInputBuffer()

 DECLARE c AS Integer

 WHILE ((c = getchar()) != '\n' AND c != EOF)

 // Keep reading characters until encountering a newline or end-of-file

 END WHILE

END FUNCTION

Declare void password (void)

START

ent_pass, count, i, x as integer

i = 3

x = 0

Print " _ _ _ "

Print " _/ ') / "

Print " / _ , o _ / / _ _ "

Print "/ _ () _ < / / < _ (_ / /) _ < / / (_ "

Print " / | "

Print " | / "

Call to the "mor" function.

print "Enter the secret code to gain access to the system"

read ent_pass

call to the wipe function

if (ent_pass = 5791) then

print "You've been granted access"

call to menu function

endif

else

system clear

print "Incorrect Secret Code...Try Again!"

print "You have three attempts"

for x=1 to 3 DO

print "Enter the secret code"

```
read ent_pass
```

```
system clear
```

```
x = x + 1
```

```
if (ent_pass == 5791) then
```

```
    print "You've been granted access"
```

```
    calls to the menu function
```

```
end if
```

```
if (ent_pass <> 5791) then
```

```
    print "Remaining Attempts:",3-x
```

```
    print "Access Status = Not Granted"
```

```
endif
```

```
endfor
```

```
endif
```

```
STOP
```

Declaring void menu (void)

START

```
x, opt as integer
```

```
x = 0
```

```
do while (opt > 1 OR opt < 5)
```

```
    Print "_____Main
Menu"
```

```
Print "  _      _      _"
```

```
Print " / )    _/_/    _/_/ )    // "
```

```
Print "/-/_ _ _ _ / _ _ , _ / /-/_ _ _ _ o// "
```

```
Print "/ ( _/_/_<_/_<_/_/_/_/_ )<_ / ( _/_/_/_<_/_<_/_ "
```

Print " / "

print "From the menu below, pick a number that corresponds with your desired option"

print "Manage Employees.....1"

print "Print Pay Slip.....2"

print "Print Report.....3"

print "If you wish to exit the system, enter 0"

read opt

system clear screen

case based on opt

case = 1:

print "You've selected option 1: Manage Employees"

call to "employ" function

case = 2:

print "You've selected option 2: Print Pay Slip"

call to "PAYSLIP" function

case =3:

print "You've selected option 3: Print Report"

call to "report" function

case = 0: call to "goodbye" function

exit (0)

default

print "Invalid Choice"

ENDCASE

endwhile

return opt

STOP

Declaring void getCurrentDate(char *date)

START

time_t t = time (NULL)

struct tm * tm = localtime (&t);

strftime (date, 20, "%Y-%m-%d", tm);

Declaring void PAYSIP (float *total_np, float *total_com, float *total_td)

START

char date [20]

call to getCurrentDate (date)

hrs_work, i, ov_hrs, units_dis as integer

bas_pay, g_p, ot, com, net_p as real

ss_tax, st_inc_tax, NHT, pen, insur, t_deduc as real

cho as string

bas_hrly_r, over_r as float

bas_hrs as integer

const bas_hrly_r = 65.00

const over_r = 1.5

const bas_hrs = 40

print "For which employee would you like to print the payslip?"

read empid

file named “employee.txt” open in read-only mode

```

if (file == NULL) then
    print “An error occurred while opening this file.”
    return;
endif

```

while reading from file until all data is processed{

```

IF (EMP [t_emp].id = empid) Then

    print “Processing for Employee ID”, emp_id[i]
    print “Enter the number of hours worked by the employee”
    read hrs_wrk

```

```

total_hw = total_hw + hrs_wrk

```

```

if (hrs_wrk > bas_hrs) then
    ov_hrs = hrs_wrk – bas_hrs
    ot = 00.00
    bas_pay = bas_hrly_r * bas_hrs
endif

```

```

else if (hrs_wrk < bas_hrs) then
    ov_hrs = 0
    ot = 00.00
    bas_pay = bas_hrly_r * bas_hrs
endif

```

```

print “Enter the number of units distributed by the employee”

```

read units_dis

total_ud = total_ud + units_dis

if (units_dis >= 200 AND units_dis < 250) then

com = 50.00

total_com = total_com + com

endif

else if (units_dis >= 250 AND units_dis < 300) then

com = 100.00

total_com = total_com + com

endif

else if (units_dis >= 300 AND units_dis < 400) then

com = 150.00

total_com = total_com + com

endif

elseif (units_dis >= 400) then

com = 200.00

total_com = total_com + com

endif

else

com = 00.00

total_com = total_com + com

endif

g_p = com + ot + bas_pay

ss_tax = g_p * 0.01

st_inc_tax = g_p * 0.07

NHT = g_p * 0.04

```

pen = g_p * 0.25
insur = g_p * 0.03

t_deduc = ss_tax + st_inc_tax + NHT + pen + insur
total_td = total_td + t_deduc

net_p = g_p - t_deduc
total_np = total_np + net_P

system clear screen

print "Would you like to print the employee's pay slip , Yes or No?"
read cho

Call to the WIPE function
    if (cho = "Yes")then

Print                "Amethyst Accueil - Distribution Firm Ltd"

Print "-----
-n"

Print                "Pay Slip"

Print "-----
\n"

Print
"Earnings          |    Hours          |    Rate          |    Current"

Print "Standard
Pay:",          |    bas_hrs          |    bas_hrly_r          |    bas_pay

Print "Overtime Pay:",          |    ov_hrs          |    over_r          |    ot

Print "Commission value:",          |          |          |    com

Print "Gross Pay:",          |          |          |    g_p

```

```
Print "-----"
--"
```

```
Print
"Deductions | Current"
```

```
Print "Social Security
Tax | ",ss_tax
```

```
Print "State Income
Tax | ",st_inc_tax
```

```
Print
"NHT | ",NHT
```

```
Print "Pension Tax | ",pen
```

```
Print "Insurance
Tax | ",insur
```

```
Print "Total
Deductions | ",t_deduc
```

```
Print "Net
Pay | ",net_p
```

```
file3 named "payslip.txt" open in append mode
```

```
if (file3 == NULL) then
```

```
    print "An error has occurred while opening this file"
```

```
    return 1
```

```
endif
```

```
writefile(file3,
```

```
"|=====
=====Date: %s", date)
```

```
writefile(file3, "|=====Amethyst Accueil - Distribution Firm
Ltd=====")
```

```

writefile(file3,
"|_____
_____")

writefile(file3, "|_____ Pay Slip for ID:
%d_____")

writefile(file3,
"|_____
_____")

writefile(file3, "|Earnings    |    Hours    |    Rate    |    Current    ")
writefile(file3, "|Standard Pay |    %d      |    %.2f    |    %.2f      ", bas_hrs,
bas_hrly_r, bas_pay)
writefile(file3, "|Standard Pay |    %d      |    %.2f    |    %.2f      ", hrs_wrk,
bas_hrly_r, bas_pay)
writefile(file3, "|Overtime Pay |    %d      |    %.2f    |    %.2f      ", ov_hrs,
over_r, ot)
writefile(file3, "|Commission   |           |           |    %.2f      ", com)
writefile(file3, "|Gross Pay    |           |           |    %.2f      ")

writefile(file3,
"|=====
=====")

writefile(file3, "|Deductions           |           Current           ")
writefile(file3, "|Social Security Tax           |           %.2f              ", ss_tax)
writefile(file3, "|State Income Tax           |           %.2f              ", st_inc_tax)
writefile(file3, "|NHT                     |           %.2f              ", NHT)
writefile(file3, "|Pension Tax             |           %.2f              ", pen)
writefile(file3, "|Insurance Tax           |           %.2f              ", insur)
writefile(file3, "|Total Deductions         |           %.2f              ", t_deduc)
writefile(file3, "|Net Pay                 |           %.2f              ", net_p)

writefile(file3,
"|_____
_____")

fclose(file3)

```

```

print "Press enter to proceed"
char c = getchar()
ELSE IF strcmp(cho, "No") == 0 THEN
    PRINT "To proceed, press enter...\n"
ELSE
    PRINT "Invalid Response...Proceeding...Press enter\n"
END IF

    getch()
END IF
END WHILE

system("pause")
WIPE ()
PRINT "\nPress the enter key to go back to the menu\n"
CHAR c = getchar()
END FUNCTION

```

Declaring void report (float total_np, float total_com, float total_td)

START

ans as string

ans as date

Call to the getCurrent Date (date) function

call to the "WIPE" function

print "Would you like to print the Report, Yes or No? "

read ans

if (ans = "Yes") then

```
print "+++++Amethyst Accueil Distributing Firm Ltd.+++++"
```

```
print"+++++"
```

```
printf("+++++R E P O R T+++++")
```

```
print "The total amount of units distributed is", total_ud
```

```
print ("The total amount of hours worked by employees is:", total_hw
```

```
printf ("The sum of net pays distributed to employees is:", total_np
```

```
printf ("The sum of commissions awarded to employees is:", total_com
```

```
printf ("The sum of deductions subtracted from Employee's Gross Pays is:
```

```
", total_td
```

```
system pause
```

```
// Creating and writing to file4
```

```
file4 named "report.txt" open in write mode
```

```
if (file4 == NULL) then
```

```
print "An error has occurred while opening file4"
```

```
return
```

```
endif
```

```
writefile(file4, "Date: %s\n", date)
```

```
writefile(file4, "Total Units Distributed: %d\n", total_ud)
```

```
writefile(file4, "Total Hours Worked: %d\n", total_hw)
```

```
writefile(file4, "Total Net Pays Distributed: %.2f\n", total_np)
```

```
writefile(file4, "Total Commissions Awarded: %.2f\n", total_com)
```

```
writefile(file4, "Total Deductions: %.2f\n", total_td)
```

```
fclose(file4)
```

```
system("pause")
```

```
endif
```

```

    WIPE() // calls to the wipe function
}

```

```

    ENDIF

```

```

END

```

Declaring employf (int *id, char*fname, char *lname, char *sex, int *acc, int *trn)

```

START

```

```

FUNCTION flushInputBuffer()

```

```

    // Flush the input buffer

```

```

    READ character FROM stdin UNTIL character = '\n' OR character = EOF

```

```

END FUNCTION

```

```

DECLARE

```

```

    employees AS employ POINTER

```

```

    numEmployees, arraySize, i, sel, search_num AS INTEGER

```

```

    foundb, found AS BOOLEAN

```

```

    c, ano AS CHARACTER

```

```

    line[MAX_LINE_SIZE] AS STRING

```

```

    arraySize = t_emp

```

```

    foundb = false

```

```

Repeat until sel equals 0:

```

```

    Print "  _____ "

```

```

    Print " / )      _/_/      _/_/ / )      // "

```

```
Print "--/ ____ _ / / _ , _ / /--/ _ . . . _ o// "
Print "/ (///<_</<_///(//)_<_ / ( ( ( (/_</<_</<_ "
Print "          /          "

Print
"_____Employee
Menu"
```

```
Print "\n _| _| _|_|_| _| _| _| _|"
Print " _|_| _|_| _| _|_| _|_| _|"
Print " _| _| _| _|_|_| _| _| _| _|"
Print " _| _| _| _| _|_| _| _|"
Print " _| _| _|_|_|_| _| _| _|_| "
```

```
Print "\n+-----+"
Print "| Select an option          |"
Print "+-----+"
Print "| 1. Display Employees          |"
Print "+-----+"
Print "| 2. Update Employee           |"
Print "+-----+"
Print "| 3. Search for an Employee     |"
Print "+-----+"
Print "| 4. Delete Employee           |"
Print "+-----+"
Print "| 5. Add Employee              |"
Print "+-----+"
Print "| 0. Return to the main menu    |"
Print "+-----+"

read sel // Scan the user's selection
```

WIPE() // Call the WIPE function to clear the input buffer

case 1:

print("Selected Option: 1...Display Employees");

Open file named "employee.txt" in read mode

IF file == NULL THEN

PRINT "An error has occurred while opening the file"

RETURN

END IF

print("Employee records:");

// Read and display each line in the file

Repeat until end of file {

Read a line from file

Print the line

}

Close file

break;

case 2:

print("Selected Option: 2...Update Employee");

Open file named "employee.txt" in read mode

IF file == NULL THEN

PRINT "Error opening file."

```

    RETURN
END IF

Open file named "copy.employee.txt" in write mode
IF file2 == NULL THEN
    PRINT "Error opening file."
    CLOSE file
    RETURN
END IF

print("Enter the ID of the Employee to update:");
read search_num;

// Loop through each record in the original file
Repeat until end of file {
    Read record from file
        IF EMP[t_emp].id == search_num THEN
            PRINT "Employee found:"
            PRINT "ID: ", EMP[t_emp].id
            PRINT "First Name: ", EMP[t_emp].fname
            PRINT "Last Name: ", EMP[t_emp].lname
            PRINT "Sex: ", EMP[t_emp].sex
            PRINT "Account Number: ", EMP[t_emp].acc
            PRINT "TRN: ", EMP[t_emp].trn

            PRINT "Enter new first name of Employee:"
            READ EMP[t_emp].fname FROM stdin
            PRINT "Enter new last name for Employee:"
            READ EMP[t_emp].lname FROM stdin

```

```

PRINT "Enter new sex of Employee:"
READ EMP[t_emp].sex FROM stdin

PRINT "Enter new Employee's Account Number:"
READ EMP[t_emp].acc FROM stdin

PRINT "Enter New Employee's Tax Registration Number:"
READ EMP[t_emp].trn FROM stdin

```

```

found = true

```

```

END IF

```

```

WRITE line TO file2

```

```

END WHILE

```

```

CLOSE file

```

```

CLOSE file2

```

```

IF found THEN

```

```

    REMOVE("employee.txt")

```

```

    RENAME("copy.employee.txt", "employee.txt")

```

```

    PRINT "Record updated successfully."

```

```

ELSE

```

```

    PRINT "Record doesn't seem to exist."

```

```

    REMOVE("copy.employee.txt")

```

```

END IF

```

```

BREAK

```

```

case 3:

```

```

    print("Selected Option: 3...Search for an Employee");

```

Open file named "employee.txt" in read mode

PRINT "Enter the employee ID to search for: "

READ search_num FROM stdin

CLEAR INPUT BUFFER

IF file == NULL THEN

 PRINT "An error has occurred while opening the file"

 RETURN

END IF

// Read and search each line in the file

Repeat until end of file {

 Read record from file

 if (EMP[t_emp].id == search_num) {

 // If employee found, print details

 print("Employee found:");

 PRINT "ID: ", EMP[t_emp].id

 PRINT "First Name: ", EMP[t_emp].fname

 PRINT "Last Name: ", EMP[t_emp].lname

 PRINT "Sex: ", EMP[t_emp].sex

 PRINT "Account Number: ", EMP[t_emp].acc

 PRINT "TRN: ", EMP[t_emp].trn

 foundb = true;

 BREAK

 END IF

END WHILE

IF NOT foundb THEN

 PRINT "Employee with ID ", search_num, " not found"

END IF

CLOSE file

BREAK

case 4:

 PRINT "Selected Option 4...Delete Employee"

 Open file named "employee.txt" in read mode

 IF file == NULL THEN

 PRINT "An error has occurred while opening the file"

 RETURN

 END IF

 Open file named "copy.employee.txt" in write mode

 IF file2 == NULL THEN

 PRINT "An error has occurred while opening the file"

 RETURN

 END IF

 print("Enter the ID of the Employee to delete:");

 read search_num;

 // Loop through each record in the original file

 Repeat until end of file {

```

Read record from file

if (EMP[t_emp].id == search_num) {
    // If the ID matches the search ID, skip writing this record to the temporary file
    print("Employee found and deleted:")
    PRINT "ID: ", EMP[t_emp].id
    PRINT "First Name: ", EMP[t_emp].fname
    PRINT "Last Name: ", EMP[t_emp].lname
    PRINT "Sex: ", EMP[t_emp].sex
    PRINT "Account Number: ", EMP[t_emp].acc
    PRINT "TRN: ", EMP[t_emp].trn
    foundb = true; // Set found flag to true
}
else
    Write record to file2

End If

End While

Close file; // Close the original file
Close file2; // Close the temporary file

if (foundb) {
    // Replace the original file with the temporary file
    If foundb Then
        Delete "employee.txt"
        Rename "copy.employee.txt" to "employee.txt"
        Print "Record deleted successfully."
    Else
        Print "Record doesn't seem to exist."
        Delete "copy.employee.txt"
    End If
}

BREAK

```

```
case 5:  
    print("Selected Option: Add Employee")  
  
    // Open file for appending  
    file named "employee.txt" open in append mode  
  
    if (file is NULL):  
        print("An error has occurred while opening this file")  
        return 1  
    endif
```

```
For i from 0 to t_emp  
    Print "Enter the employee's ID:"  
    Read EMP[i].id  
    Clear input buffer  
  
    Print "Enter the employee's first name:"  
    Read EMP[i].fname  
    Clear input buffer  
  
    Print "Enter the employee's last name:"  
    Read EMP[i].lname  
    Clear input buffer  
  
    Print "Enter the employee's sex:"  
    Read EMP[i].sex  
    Clear input buffer
```

Print "Enter the employee's account number:"

Read EMP[i].acc

Clear input buffer

Print "Enter the employee's TRN:"

Read EMP[i].trn

Clear input buffer

Write to file: "%d,%s,%s,%c,%d,%d\n",

EMP[i].id,

EMP[i].fname,

EMP[i].lname,

EMP[i].sex,

EMP[i].acc,

EMP[i].trn

Print "Employee record successfully stored"

// Ask if the user wants to add another employee

IF i < t_emp - 1 THEN

Declare choice as char

Print "Do you want to add another employee? (y/n):"

Read choice

If choice is not 'y' and choice is not 'Y' Then

BREAK

Exit loop

End If

End If

End For

Close file

BREAK

case = 0: call to “menu” function

BREAK

DEFAULT: print “Invalid Choice”

BREAK

END SWITCH

END REPEAT

END

// Function prototype Declarations

void welcome();

void mor();

void password();

void WIPE();

void menu();

void employf();

void report(float total_np, float total_com, float total_td);

void mor_alt_BG();

void goodbye();

void PAYSLIP(float *total_np, float *total_com, float *total_td);

START

int main():

```
// Call to the welcome function
```

```
welcome()
```

```
// Prompt to continue
```

```
print("Press enter to continue...waiting")
```

```
char c = getchar()
```

```
// Call to the WIPE function
```

```
WIPE()
```

```
// Call to the mor function
```

```
mor()
```

```
// Call to the password function
```

```
password()
```

```
// Call to the WIPE function
```

```
WIPE()
```

```
STOP
```

Data Structures and Records

Employ is a record data type structure (typedef) that stores various personal information of the employees of the firm. This data structure is accompanied by an array called **EMP**, that is, there is the implementation of a structure of an array, entitled “**employ EMP [t_emp]**”, with an array (t_emp) of size 6. This structure of an array was implemented mainly in the **employf** function, due to it being a function that maintains employee management, but also made appearances in the **payslip** function, so as for ease of cross referencing.

Structuring employee data within an array streamlines organization, with each element representing an employee and attributes delineating specific information like ID and name. As employee numbers increase, array usage simplifies management processes, enabling easy addition or removal of employees. Array-based iteration facilitates streamlined operations across all employees or specific subsets, enhancing efficiency and scalability in data manipulation tasks.

Attribute	Data Type	Purpose
Id	integer	An integer field utilized to assign a unique identification number to each employee, facilitating efficient data retrieval and management.
Fname	String	A character array of size 50, intended to store the first name of the employee for personal identification and reference.
Lname	string	A character array of size 50, designated for storing the last name of the employee for personal identification and reference.
Sex	Character	A character variable employed to denote the gender or sex of the employee.
Acc	Integer	To store information related to the employee's account, potentially used for financial or administrative purposes.
Trn	Integer	To store information related to the employee's government issued tax registration number.

Source Code - C Code

```
1  /*
2  Programmer: Robyn Sterling of 602-B
3
4  Course: CS
5
6  Format: This program is a combination of a database and an accounting styling format.
7
8  Description: It will allow the user to manage a set of employees and records, given the ability to do
9  things from searching, display, adding, removing and updating...the user will also have the ability to
compute
10 an individual's customized pay slip as well as a company's report, by inputting a minimal amount of
data. These
11 pieces of data will all be recorded in separate files, for ease of data allocation.
12
13 Program Start Date: November 24, 2022
14
15 Program End Date: April 7, 2024
16 */
17
18 //Libraries
19 #include <stdlib.h>
20 #include <stdio.h>
21 #include <conio.h>
22 #include <string.h>
23 #include <stdbool.h>
24 #include <time.h>
25
26 //Constants
27 #define t_emp 6//defining size of primary array
28 #define MAX_LINE_SIZE 100 // Maximum size of a line in the file
29
30 //declaration and initialization of variables
31 typedef struct
32 {
33     int id;
34     char fname[50];
35     char lname[50];
```

```

36 char sex;
37 int acc;
38 int trn;
39 } employ;
40
41 //Global Variables
42 int total_ud, total_hw;//due to the simplicity of these variables, data leakages are not at risk
43 float total_np, total_com, total_td;//due to the simplicity of these variables, data leakages are not
at risk
44 FILE *file, *file2, *file3, *file4;//declaring three files
45 //declaring elements of the structure, "employ" to be set in the array, "EMP" with an array size of,
"t_employ".
46 employ EMP [t_employ];
47
48
49 //Algorithm/Function for color graphic
50 void mor () {
51     printf("\033[0;35m");
52 }
53
54 //Algorithm/Function for color graphic.2 and BG color graphic
55 void mor_alt_BG (){
56     printf ("\e[1;95m");
57     printf ("\e[47m");
58 }
59
60 //Algorithm that welcomes the user to the system
61 void welcome (){
62     mor_alt_BG ();//calls to the background graphic function
63     printf ("ĚšĚš Welcome to the Amethyst Accueil System ĚšĚš\n");
64     printf("\n");
65     printf(" // \\\ \ | | \\\ \ / | | | | _ _ | | | | _ _ | | \\\ \ / (( \\\ | | | | \n");
66     printf(" || = || | | \\\ / | | || ==    | |   || = || ) /   \\\ \    | |   \n");
67     printf(" | | | | | |   | | | | _ _    | |   | |   | | //   \\\_) )    | |   \n");
68     printf("\n");
69     printf(" // \\\ \   //      //      | | | | | |   | | | |   \n");
70     printf(" || = || ((      ((      | | | | || == | | | |   \n");
71     printf(" | | | | \\\ \ _ _   \\\ \ _ _ \\\ \ _ // | | _ _ | | | | _ _   \n");
72     printf("\n");
73
74     printf("\n");

```

```

75     printf("      (|  \ \      \n");
76     printf("      |      |  \n");
77     printf("      |      ||/  \n");
78     printf("CASA  (/ \__ / |__ /  PIEDRAS PRECIOSAS\n");
79
80
81
82 }
83
84 //Algorithm that wishes the user off the system
85 void goodbye () {
86     mor_alt_BG (); //calls to the background graphic function
87     printf ("Thank you for using the Amethyst Accueil Distribution Firm Ltd system...Goodbye!");
88
89     printf("
90 printf(" // \\\ \ | \\\ \ / | | | | | | | | | | \\\ \ / ( ( \ \ | | | | \n");
91 printf(" || = || | | \ \ / | | | = | | | | = | | ) / \ \ \ \ | | \n");
92 printf(" | | | | | | | | | | | | | | | | | | | | | | \n");
93 printf("
94 printf(" // \\\ \ // // | | | | | | | | | | \n");
95 printf(" || = || ( ( ( ( | | | | | = | | | | \n");
96 printf(" | | | | \\\ \_ \\\ \_ \\\ \_ / | | | | | | | | \n");
97 printf("
98
99     printf("
100    printf("      (|  \ \      \n");
101    printf("      |      |  \n");
102    printf("      |      ||/  \n");
103    printf("CASA  (/ \__ / |__ /  PIEDRAS PRECIOSAS\n");
104 }
105 //Algorithm/Function that clears screen
106 void WIPE () {
107     system ("cls"); //implements a clear screen
108 }
109
110 void clearInputBuffer() {
111     int c;
112     while ((c = getchar()) != '\n' && c != EOF);
113 }
114
115 //Algorithm/Function for Passcode implementation

```

```

116 void password(void)
117 { //begins the password function
118
119     //declaration and initialization of variables
120     int ent_pass, count;
121     int i=3;
122     int x = 0;
123
124
125     printf("                                \n");
126     printf(" _//_          ' _ ) _//_          \n");
127     printf(" /   _ _ ,    o   /   /   \n");
128     printf("/ _ _ ( ) ( ) < _ / / < _ ( _//_ ) < / _ ( _\n");
129     printf("      /|         \n");
130     printf("      |/\n");
131
132     mor();//implements text color as purple
133     printf ("Enter the secret code to gain access to the system\n");//prompts the user to enter a
secret code
134     scanf ("%d", &ent_pass);//reads the secret code entered
135     WIPE ();//calls to the WIPE function
136
137     if (ent_pass == 5791)
138     { //begins the if statement
139         printf ("You've been granted access\a\n");//outputs a success message
140         menu();//calls to the menu function
141     } //ends the if statement
142
143     else
144     { //begins the else of the if statement
145         WIPE ();//calls to the WIPE function
146         printf ("Incorrect Secret Code...Try Again!\n");//outputs a failure message
147         printf ("You have three attempts\n");//outputs the amount of attempts the user has to gain
access to the system
148
149         for (x = 1;x <= 3 && ent_pass != 5791; x++)//gives the user three attempts to enter the
system through loop
150             { //begins the for loop
151
152                 printf ("\nRe-enter the secret code\n");//prompts the user to re-enter the secret code
153                 scanf ("%d", &ent pass);//reads the entered secret code

```

```

154         WIPE (); //calls to the WIPE function
155
156         if(ent_pass == 5791) //if this condition is determined to be true, the user gains
entry into the system
157             { //begins the if statement
158                 printf ("You've been granted access\n"); //prompts success message
159                 menu(); //calls to the menu function after entering the correct code
160             } //ends the if statement
161
162         if(ent_pass != 5791) //if this condition is determined to be true, the user is
denied of entry into the system
163             { //begins the if statement
164                 printf ("Remaining Attempts: %d \n", 3-x); //calculates the amount of attempts the
user has left after each loop
165                 printf ("Access Status = Not Granted\n"); //displays the user with their periodic
access status
166             } //ends the if statement
167
168         } //ends the for loop
169
170     } //ends the else of the if statement
171
172 } //ends the password function
173
174 //Algorithm/Function that prompts menu
175 void menu()
176 { //begins the menu function
177
178     //declaration and initialization of variables
179     int x = 0, opt;
180
181     do
182     { //begins do...while loop
183
184
185         printf ("_____Main Menu\n");
186         printf ("_____ \n");
187         printf (" / ) _____ / - / ) _____ // \n");
188         printf (" /--/ _____ / - / ) /--/ . . . o // \n");
189         printf (" / ( / / / < < / < / / ( / - ) < / ( ( ( ( / < / < / \n");
190         printf (" / _____ / \n");

```

```

191
192
193
194     //prints the main menu with options to the user
195     printf ("\nFrom the menu below, pick a number that corresponds with your desired option\n");
196     printf ("Manage Employees.....1\n");
197     printf ("Print Pay Slip.....2\n");
198     printf ("Print Report.....3\n");
199     printf ("If you wish to exit the system, enter 0\n\n");
200     scanf ("%d", &opt); //reads the option inputted by the user
201     WIPE (); //calls to the WIPE function
202
203     switch (opt)
204     { //begin of switch statement
205
206         case 1: printf ("You've selected option 1: Manage Employees\n"); //outputs the option
the user had selected
207         employf(); //calls to the manage employee function
208         break;
209
210         case 2: printf ("You've selected option 2: Print Pay Slip\n"); //outputs the option the
user had selected
211         PAYSLIP(); //calls to the payslip function
212         break;
213
214         case 3: printf ("You've selected option 3: Print Report\n"); //outputs the option the
user had selected
215         report(); //calls to the report function
216         break;
217
218         case 0: goodbye(); //calls to the goodbye function
219                 exit (0);
220
221         default: printf ("Invalid Choice\n");
222     } //end of switch statement
223
224
225     } while (opt >1 || opt <5); //end of do...while loop; this iteration will repeatedly prompt the menu
until the user decides to exit the system with the terminator 0
226
227     return opt; //returns the value of the variable "opt"

```

```

228
229 }//end of the menu function
230
231 // getCurrentDate Function Definition - Gets the current date of Report Production
232 void getCurrentDate(char *date) {
233     time_t t = time(NULL);
234     struct tm *tm = localtime(&t);
235     strftime(date, 20, "%Y-%m-%d", tm);
236 }
237
238
239 //Payslip Function Definition - Prints Pay Slip of Each Employee
240 void PAYSIP ()
241 { //begin of payslip function
242     char date[20];
243     getCurrentDate(date);
244
245
246     //declaration and initialization of variables
247     int i, hrs_wrk, ov_hrs, units_dis, empid;
248     float bas_pay, g_p, ot, com, net_p;
249     float ss_tax, st_inc_tax, NHT, pen, insur, t_deduc;
250     char cho [10];
251     const float bas_hrly_r = 65.00;
252     const int bas_hrs = 40;
253     const float over_r = 1.5;
254
255     printf ("For which employee would you like to print the payslip?\n");
256     scanf ("%d", &empid);
257
258     file = fopen ("employee.txt", "r");
259     if (file == NULL){
260         printf ("An error occurred while opening the file. \n");
261         return;
262     }
263
264     while (fscanf(file, "%d,%49[^\,],%49[^\,],%c,%d,%d\n", &EMP[t_emp].id, EMP[t_emp].fname, EMP[t_emp]
265     .lname, &EMP[t_emp].sex, &EMP[t_emp].acc, &EMP[t_emp].trn) == 6) {
266         if (EMP[t_emp].id == empid) { // If the ID matches the search ID, update the record
267             printf("\nProcessing for Employee ID: %d\n", EMP[t_emp].id); //displays to the user the ID for

```

```

which the payslip is being generated for
268     printf ("Enter the number of hours worked by the employee\n");//prompts the user to enter the
amount of hours worked by the employee
269     scanf ("%d", &hrs_wrk);
270     total_hw = total_hw + hrs_wrk;
271
272     if (hrs_wrk > bas_hrs)//this condition of the if statement is based on the assumption that the
worker had worked overtime
273     {
274         ov_hrs = hrs_wrk - bas_hrs;
275         ot = ov_hrs * over_r * bas_hrly_r;
276         bas_pay = bas_hrly_r * bas_hrs;
277     }
278     else if (hrs_wrk == bas_hrs)//this condition of the if statement is based on the assumption
that the worker had worked no overtime
279     {
280         ov_hrs = hrs_wrk - bas_hrs;
281         ot = 00.00;
282         bas_pay = bas_hrly_r * bas_hrs;
283     }
284     else if (hrs_wrk < bas_hrs)//this condition of the if statement is based on the assumption
that the worker had worked below the average work hours
285     {
286         ov_hrs = 0;
287         ot = 00.00;
288         bas_pay = bas_hrly_r * hrs_wrk;
289     }
290
291     fflush(stdin);
292
293     printf ("Enter the number of units distributed by the employee\n");//prompts the user to enter
the amount of units the worker had distributed
294     scanf ("%d", &units_dis);
295     total_ud = total_ud + units_dis;
296
297     //Calculate Commission
298     if (units_dis >= 200 && units_dis <250)//this condition rewards the worker with the lowest
amount of commission, if they distributed between 200 and 249
299     {
300         com = 50.00;
301         total_com = total_com + com;

```

```

302     }
303     else if(units_dis >=250 && units_dis<300)//this condition rewards the worker with the
second lowest amount of commission, if they distributed between 250 and 299
304     {
305         com = 100.00;
306         total_com = total_com + com;
307     }
308     else if (units_dis >=300 && units_dis<400)//this condition rewards the worker with the
third lowest amount of commission, if they distributed between 300 and 399
309     {
310         com = 150.00;
311         total_com = total_com + com;
312     }
313     else if (units_dis >= 400)//this condition rewards the worker with the highest
amount of commission, if they distributed anything higher or equivalent to 400
314     {
315         com = 200.00;
316         total_com = total_com + com;
317     }
318     else //this condition rewards the worker with no commission, if no other
conditions were met
319     {
320         com = 00.00;
321         total_com = total_com + com;
322     }
323
324     g_p = com + ot + bas_pay;//calculates the gross pay of the employee
325
326     //calculates the deductions
327     ss_tax = g_p * 0.01;
328     st_inc_tax = g_p * 0.07;
329     NHT = g_p * 0.04;
330     pen = g_p * 0.25;
331     insur = g_p * 0.03;
332     t_deduc = ss_tax + st_inc_tax + NHT + pen + insur;//sums up the total amount of deductions
333     total_td = total_td + t_deduc;//totals the total deductions of each employee
334
335     net_p = g_p - t_deduc;//calculates the net pay of the employee
336     total_np = total_np + net_p;//totals the net pay of each employee
337     WIPE ()//calls to the WIPE function
338

```

```

339     printf ("\nWould you like to print the employee's pay slip, Yes or No?\n");
340     scanf ("%s", cho);
341
342     WIPE ();
343     if (strcmp (cho, "Yes")==0)
344     {
345         printf ("Pay Slip being created...wait...\n");
346         sleep (5); //pauses the system for roughly 5 seconds
347         printf ("\n|=====Amethyst Accueil - Distribution Firm
Ltd=====|");
348         printf
349         ("|_____|Pay Slip for ID:
%d_____|", EMP[t_emp].id);
350         printf
351         ("|_____|");
352         printf ("\n|Earnings      |      Hours      |      Rate      |
Current      \n");
353         if (hrs_wrk > bas_hrs){
354             printf ("\n|Standard Pay      |      %d      |      %.2f      |
%.2f      \n", bas_hrs,bas_hrly_r,bas_pay);
355         }
356         else{
357             printf ("\n|Standard Pay      |      %d      |      %.2f      |
%.2f      \n", hrs_wrk,bas_hrly_r,bas_pay);
358         }
359         printf ("\n|Overtime Pay      |      %d      |      %.2f      |
%.2f      \n", ov_hrs, over_r,ot);
360         printf ("\n|Commission      |      |      |
%.2f      \n", com);
361         printf ("\n|Gross Pay      |      |      |
%.2f      \n", g_p);
362         printf
363         ("|=====|");
364         printf ("\n|Deductions      |      Current
\n");
365         printf ("\n|Social Security Tax      |      %.2f
\n", ss_tax);
366         printf ("\n|State Income Tax      |      %.2f
\n", st_inc_tax);
367         printf ("\n|NHT      |      %.2f
\n");

```

```

\n", NHT);
366         printf ("\n|Pension Tax                |                |                |                |\n", pen);
367         printf ("\n|Insurance Tax                 |                |                |                |\n", insur);
368         printf ("\n|Total Deductions              |                |                |                |\n", t_deduc);
369         printf ("\n|Net Pay                       |                |                |                |\n", net_p);
370         printf
(("\n|_____ \n");
371
372         file3 = fopen ("payslip.txt", "a");
373         if (file3 == NULL)
374         {
375             printf ("An error has occurred while opening this file\n");
376             return 1;
377         }
378
379         fprintf (file3,
"\n|=====Date: %s\n", date);
380         fprintf (file3, "\n|=====Amethyst Accueil - Distribution Firm
Ltd===== \n");
381         fprintf (file3,
"\n|_____ \n");
382         fprintf (file3, "\n|_____Pay Slip for ID:
%d_____ \n", EMP[t_emp].id);
383         fprintf (file3,
"\n|_____ \n");
384         fprintf (file3, "\n|Earnings      |          Hours      |          Rate      |
Current\n");
385         fprintf (file3, "\n|Standard Pay |          %d          |          %.2f       |
%.2f\n", bas_hrs,bas_hrly_r,bas_pay);
386         fprintf (file3, "\n|Standard Pay |          %d          |          %.2f       |
%.2f\n", hrs_wrk,bas_hrly_r,bas_pay);
387         fprintf (file3, "\n|Overtime Pay |          %d          |          %.2f       |
%.2f\n", ov_hrs, over_r,ot);
388         fprintf (file3, "\n|Commission   |                    |                    |
%.2f\n", com);
389         fprintf (file3, "\n|Gross Pay    |                    |                    |
%.2f\n", g_p);

```

```

390             fprintf (file3,
"\n|===== \n");
391             fprintf (file3, "\n|Deductions | Current
\n");
392             fprintf (file3, "\n|Social Security Tax | %.2f
\n", ss_tax);
393             fprintf (file3, "\n|State Income Tax | %.2f
\n", st_inc_tax);
394             fprintf (file3, "\n|NHT | %.2f
\n", NHT);
395             fprintf (file3, "\n|Pension Tax | %.2f
\n", pen);
396             fprintf (file3, "\n|Insurance Tax | %.2f
\n", insur);
397             fprintf (file3, "\n|Total Deductions | %.2f
\n", t_deduc);
398             fprintf (file3, "\n|Net Pay | %.2f
\n", net_p);
399             fprintf (file3,
"\n|_____ \n");
400
401             fclose (file3);
402
403             printf ("Press enter to proceed\n");
404             char c = getchar();
405             }
406             else if (strcmp (cho, "No")==0)
407             {
408                 printf ("To proceed, press enter...\n");
409             }
410
411             else
412             {
413                 printf ("Invalid Response...Proceeding...Press enter\n");
414             }
415             getch();
416         }
417     }
418     system ("pause");//as the name suggests, pauses the system,act as a break
419     WIPE ();
420     printf ("\nPress the enter key to go back to the menu\n");

```

```

421     char c = getchar();
422 }
423
424 //Report Function Definition - Prints the report
425 void report (){
426
427     char ans [10];
428     char date[20];
429     getCurrentDate(date);
430
431     WIPE ();
432     printf ("Would you like to print the Report, Yes or No?\n");
433     scanf ("%s", &ans);
434
435     if (strcmp(ans, "Yes")==0){
436         printf ("+++++=Amethyst Accueil Distributing Firm
Ltd.+++++\n");
437         printf
("+++++\n");
438         printf ("+++++R E P O R
T+++++\n");
439         printf ("The total amount of units distributed is
%d\n", total_ud);
440         printf ("The total amount of hours worked by employees is:
%d\n", total_hw);
441         printf ("The sum of net pays distributed to employees is:
%.2f\n", total_np);
442         printf ("The sum of commissions awarded to employees is:
%.2f\n", total_com);
443         printf ("The sum of deductions subtracted from Employee's Gross Pays is:
%.2f\n", total_td);
444         system ("pause");
445
446         // Creating and writing to file4
447         FILE *file4 = fopen("report.txt", "w");
448         if (file4 == NULL) {
449             printf("An error has occurred while opening file4\n");
450             return;
451         }
452
453         fprintf(file4, "Date: %s\n", date);

```

```

454     fprintf(file4, "Total Units Distributed: %d\n", total_ud);
455     fprintf(file4, "Total Hours Worked: %d\n", total_hw);
456     fprintf(file4, "Total Net Pays Distributed: %.2f\n", total_np);
457     fprintf(file4, "Total Commissions Awarded: %.2f\n", total_com);
458     fprintf(file4, "Total Deductions: %.2f\n", total_td);
459     fclose(file4);
460
461     system ("pause");
462 }
463 WIPE();//calls to the wipe function
464
465 }
466
467
468 void employf( int *id, char *fname, char *lname, char *sex,int *acc, int *trn)
469 {
470
471     fflush (stdin);
472
473     employ* employees;
474     int numEmployees = 0;
475     int arraySize = t_emp;
476     int i, sel, search_num;
477     bool foundb = false;
478     int found = 0;
479     char c;
480     char ano [10];
481     char line[MAX_LINE_SIZE];
482
483
484     do{//the menu2.0 function prompt resides here; it stems off of the first option being "Manage
Employees"
485
486         printf("
487         printf(" / )
488         printf(" /--/
489         printf("/ ( / / / < < / < < / / / ( / / / ) < / ( ( ( ( / < / < < /
490         printf("
491         printf ("
492
493

```

```

494         printf("\n
495         printf("
496         printf("
497         printf("
498         printf("
499
500
501         printf("\n");
502         printf("+-----+\n");
503         printf("| Select an option | \n");
504         printf("+-----+\n");
505         printf("| 1. Display Employees | \n");
506         printf("+-----+\n");
507         printf("| 2. Update Employee | \n");
508         printf("+-----+\n");
509         printf("| 3. Search for an Employee | \n");
510         printf("+-----+\n");
511         printf("| 4. Delete Employee | \n");
512         printf("+-----+\n");
513         printf("| 5. Add Employee | \n");
514         printf("+-----+\n");
515         printf("| 0. Return to the main menu | \n");
516         printf("+-----+\n");
517         scanf ("%d", &sel); //scans the selection the user inputs after being shown menu2.0
518         WIPE (); //calls to the WIPE function
519
520         switch (sel)
521         {
522         case 1: printf ("Selected Option: 1...Display Employees\n");
523
524                 file = fopen("employee.txt", "r"); // Open file in read mode
525
526                 if (file == NULL) {
527                         printf("An error has occurred while opening the file\n");
528                         return 1;
529                 }
530
531                 printf("Employee records:\n");
532
533                 // Read and display each line in the file
534                 while (fgets(line, MAX LINE SIZE, file) != NULL) {

```

```

535         printf("%s", line);
536     }
537
538     fclose(file);
539     break;
540
541     case 2: printf ("Selected Option 2: Update Employee\n");
542
543     file = fopen("employee.txt", "r"); // Open original file in read mode
544     if (file == NULL) {
545         printf("Error opening file.\n");
546         return;
547     }
548
549     file2 = fopen("copy.employee.txt", "w"); // Open temporary file in write mode
550     if (file2 == NULL) {
551         printf("Error opening file.\n");
552         fclose(file);
553         return;
554     }
555
556     printf("\nEnter the ID of the Employee to update: \n");
557     scanf("%d", &search_num);
558
559     // Loop through each record in the original file
560     while (fscanf(file, "%d,%49[^\n],%49[^\n],%c,%d,%d\n", &EMP[t_emp].id,
EMP[t_emp].fname, EMP[t_emp].lname, &EMP[t_emp].sex, &EMP[t_emp].acc, &EMP[t_emp].trn) == 6) {
561         if (EMP[t_emp].id == search_num) { // If the ID matches the search ID, update
the record
562
563             printf("Employee found:\n");
564             printf("ID: %d\nFirst Name: %s\nLast Name: %s\nSex: %c\nAccount Number:
%d\nTRN: %d\n",
565                 EMP[t_emp].id, EMP[t_emp].fname, EMP[t_emp].lname, EMP[t_emp].sex,
EMP[t_emp].acc, EMP[t_emp].trn);
566
567             printf("Enter new first name of Employee:\n");
568             scanf(" %49s", EMP[t_emp].fname);
569             printf("Enter new last name for Employee:\n");
570             scanf(" %49s", EMP[t_emp].lname);
571             printf("Enter new sex of Employee:\n");

```

```

572             scanf(" %c", &EMP[t_emp].sex);
573             printf("Enter new Employee's Account Number:\n");
574             scanf("%d", &EMP[t_emp].acc);
575             printf("Enter New Employee's Tax Registration Number:\n");
576             scanf("%d", &EMP[t_emp].trn);
577             found = true; // Set found flag to true
578         }
579         fprintf(file2, "%d,%s,%s,%c,%d,%d\n",EMP[t_emp] .id, EMP[t_emp].fname,
EMP[t_emp].lname, EMP[t_emp].sex, EMP[t_emp].acc, EMP[t_emp].trn);
580     }
581
582     fclose(file); // Close the original file
583     fclose(file2); // Close the temporary file
584
585     if (found) {
586         // Replace the original file with the temporary file
587         remove("employee.txt");
588         rename("copy.employee.txt", "employee.txt");
589         printf("Record updated successfully.\n");
590     } else {
591         printf("Record doesn't seem to exist.\n");
592         remove("copy.employee.txt"); // Remove the temporary file
593     }
594
595     break;
596
597     case 3:printf ("Selected Option 3...Search for an Employee\n");
598
599         file = fopen("employee.txt", "r");//opens original file in read mode
600
601         printf("Enter the employee ID to search for: ");
602         scanf("%d", &search_num);
603         clearInputBuffer();
604
605         if (file == NULL) {
606             printf("An error has occurred while opening the file\n");
607             return;
608         }
609
610         // Read and search each line in the file
611         while (fgets(line, sizeof(line), file) != NULL) {

```

```

612             sscanf(line, "%d,%49[^,],%49[^,],%c,%d,%d", &EMP[t_emp].id, EMP[t_emp].fname,
EMP[t_emp].lname, &EMP[t_emp].sex, &EMP[t_emp].acc, &EMP[t_emp].trn);
613             if (EMP[t_emp].id == search_num) {
614                 printf("Employee found:\n");
615                 printf("ID: %d\nFirst Name: %s\nLast Name: %s\nSex: %c\nAccount Number:
%d\nTRN: %d\n",
616                     EMP[t_emp].id, EMP[t_emp].fname, EMP[t_emp].lname, EMP[t_emp].sex,
EMP[t_emp].acc, EMP[t_emp].trn);
617                 foundb = true;
618                 break;
619             }
620         }
621
622         if (!foundb) {
623             printf("Employee with ID %d not found\n", id);
624         }
625
626         fclose(file);
627
628         break;
629
630
631     case 4: printf ("Selected Option 4...Delete Employee\n");
632
633         file = fopen("employee.txt", "r"); // Open original file in read mode
634         if (file == NULL) {
635             printf("Error opening file.\n");
636             return;
637         }
638
639         file2 = fopen("copy.employee.txt", "w"); // Open temporary file in write mode
640         if (file2 == NULL) {
641             printf("Error opening file.\n");
642             fclose(file);
643             return;
644         }
645
646         printf("Enter the ID of the Employee to delete: \n");
647         scanf("%d", &search_num);
648
649         // Loop through each record in the original file

```

```

650         while (fscanf(file, "%d,%49[^\n],%49[^\n],%c,%d,%d\n", &EMP[t_emp].id,
EMP[t_emp].fname, EMP[t_emp].lname, &EMP[t_emp].sex, &EMP[t_emp].acc, &EMP[t_emp].trn) == 6) {
651             if (EMP[t_emp].id == search_num) { // If the ID matches the search ID, skip
writing this record to the temporary file
652                 printf("Employee found and deleted:\n");
653                 printf("ID: %d\nFirst Name: %s\nLast Name: %s\nSex: %c\nAccount Number:
%d\nTRN: %d\n",
654                     EMP[t_emp].id, EMP[t_emp].fname, EMP[t_emp].lname, EMP[t_emp].sex,
EMP[t_emp].acc, EMP[t_emp].trn);
655                 foundb = true; // Set found flag to true
656             } else {
657                 fprintf(file2, "%d,%s,%s,%c,%d,%d\n", EMP[t_emp].id, EMP[t_emp].fname,
EMP[t_emp].lname, EMP[t_emp].sex, EMP[t_emp].acc, EMP[t_emp].trn);
658             }
659         }
660
661         fclose(file); // Close the original file
662         fclose(file2); // Close the temporary file
663
664         if (foundb) {
665             // Replace the original file with the temporary file
666             remove("employee.txt");
667             rename("copy.employee.txt", "employee.txt");
668             printf("Record deleted successfully.\n");
669         } else {
670             printf("Record doesn't seem to exist.\n");
671             remove("copy.employee.txt"); // Remove the temporary file
672         }
673
674         break;
675
676
677         case 5: printf ("Selected Option: 5...Add Employee\n");
678
679             file = fopen("employee.txt", "a"); // Open file in append mode
680
681             if (file == NULL) {
682                 printf("An error has occurred while opening this file\n");
683                 return 1;
684             }
685

```

```

686     for (int i = 0; i < t_emp; i++) {
687         printf("Enter the employee's ID: \n");
688         scanf("%d", &EMP[i].id);
689         clearInputBuffer();
690
691         printf("Enter the employee's first name: \n");
692         scanf("%s", EMP[i].fname);
693         clearInputBuffer();
694
695         printf("Enter the employee's last name: \n");
696         scanf("%s", EMP[i].lname);
697         clearInputBuffer();
698
699         printf("Enter the employee's sex: \n");
700         scanf(" %c", &EMP[i].sex);
701         clearInputBuffer();
702
703         printf("Enter the employee's account number: \n");
704         scanf("%d", &EMP[i].acc);
705         clearInputBuffer();
706
707         printf("Enter the employee's TRN: \n");
708         scanf("%d", &EMP[i].trn);
709         clearInputBuffer();
710
711         fprintf(file, "%d,%s,%s,%c,%d,%d\n",
712                 EMP[i].id,
713                 EMP[i].fname,
714                 EMP[i].lname,
715                 EMP[i].sex,
716                 EMP[i].acc,
717                 EMP[i].trn);
718
719         printf("Employee record successfully stored\n");
720
721         // Ask if the user wants to add another employee
722         if (i < t_emp - 1) {
723             char choice;
724             printf("Do you want to add another employee? (y/n): ");
725             scanf(" %c", &choice);
726             if (choice != 'y' && choice != 'Y')

```

```

727             break;
728         }
729     }
730
731     fclose(file);
732
733
734     break;
735
736     case 0: menu();//takes the user back to the menu function
737     break;
738
739     default: printf ("Invalid Choice\n");//prints error message, if the user inputs any other
option that didn't exist on the menu
740     break;
741
742     }//end switch selection
743
744     }while (sel>1 || sel<5);//end of do...while loop for menu
745 }
746
747 // Function prototype Declaration
748 void welcome();
749 void mor();
750 void password();
751 void WIPE();
752 void menu();
753 void employf();
754 void report ();
755 void mor_alt_BG();
756 void goodbye();
757 void PAYSZIP();
758
759 int main ()
760
761 { //begin of driver function
762     welcome();//calls to the welcome function
763     printf ("\nPress enter to continue...waiting\n");
764     char c = getchar();
765     WIPE ()//calls to the WIPE function
766

```

```
767     mor ();//calls to color function
768     password();//calls to password function
769     WIPE();//calls to clear screen function
770 }//end of driver function
771
772
```

Test Plan

Test Number	Data Type	Data Name	Test Data	Purpose	Expected Result	Actual Results
1	integer	ent_pass	5791	To store the password inputted by the user into the variable, that will determine whether the user gains access or not.	Access is Granted.	Access is Granted
2	integer	ent_pass	5970	To store the password inputted by the user into the variable, that will determine whether the user gains access or not.	Access is Granted.	Access is not Granted. The password doesn't match with that of the systems. The user is instead given 3 more attempts before the program terminates.
3	integer	ent_pass	word	To store the password inputted by the user into the variable, that will determine whether the user gains access or not.	Access is Granted	Access is not granted, rather the program terminates, as the inputted data doesn't match with its designated data type.
4	integer	ent_pass	0.223	To store the password	Access is granted.	Access is not granted,

				inputted by the user into the variable, that will determine whether the user gains access or not.		rather the program terminates, as the inputted data doesn't match with its designated data type.
5	integer	opt	1	To store the option inputted by the user into the variable and determine the output.	Gives the user the opportunity to manage employees.	Gives the user the opportunity to manage employees.
6	integer	opt	2	To store the option inputted by the user into the variable and determine the output.	Gives the user the opportunity to print the pay slip.	Gives the user the opportunity to print the pay slip.
7	integer	opt	3	To store the option inputted by the user into the variable and determine the output.	Gives the user the opportunity to print the report.	Gives the user the opportunity to print the report.
8	integer	opt	0	To store the option inputted by the user into the variable and determine the output.	Gives the user the opportunity to exit the system.	Gives the user the opportunity to exit the system.
9	integer	opt	5	To store the option inputted by the user into	Gives the user the opportunity to manage	The user is shown the message "Invalid

				the variable and determine the output.	employees or print the pay slip or print report or exit the system.	Choice” and prompted to select an option again.
10	integer	opt	word	To store the option inputted by the user into the variable and determine the output.	Gives the user the opportunity to manage employees or print the pay slip or print report or exit the system.	Exits the system, because the data entered doesn’t match with the data type designated to the data storage.
11	integer	opt	0.222	To store the option inputted by the user into the variable and determine the output.	Gives the user the opportunity to manage employees or print the pay slip or print report or exit the system.	Exits the system, because the data entered doesn’t match with the data type designated to the data storage.
12	Integer	hrs_wrk	40	To store the hours worked by the employee inputted by the user into the variable to determine calculations.	Accepts the data, proceeds to precise calculations.	Accepts the data, proceeds to precise calculations.
13	integer	hrs_wrk	24.5	To store the hours worked by the employee inputted by the user into the variable	Accepts the data, proceeds to precise calculations.	Accepts the whole number of the decimal and ignores the decimals and proceeds to

				to determine calculations.		precise calculations.
14	Integer	hrs_wrk	word	To store the hours worked by the employee inputted by the user into the variable to determine calculations.	Accepts the data, proceeds to precise calculations.	Processes and Outputs null/waste calculations as the data doesn't match with the designated data type.
15	integer	units_dis	200	To store the units distributed by the employee inputted by the user into the variable to determine calculations.	Accepts the data, proceeds to precise calculations.	Accepts the data, proceeds to precise calculations.
16	integer	units_dis	456.890	To store the units distributed by the employee inputted by the user into the variable to determine calculations.	Accepts the data, proceeds to precise calculations.	Accepts the whole number of the decimal and ignores the decimals and proceeds to precise calculations.
17	Integer	units_dis	word	To store the units distributed by the employee inputted by the user into the variable to determine calculations.	Accepts the data, proceeds to precise calculations.	Processes and Outputs null/waste calculations as the data doesn't match with the designated data type.
18	char[]	cho	Yes	Stores the data into the variable to determine	The Pay Slip is displayed so the screen.	The Pay Slip is displayed so the screen.

				whether to display the pay slip or not.		
19	char[]	cho	No	Stores the data into the variable to determine whether to display the pay slip or not.	The Pay Slip is not displayed to the screen.	The Pay Slip is not displayed to the screen.
20	char[]	cho	Maybe	Stores the data into the variable to determine whether to display the pay slip or not.	The Pay Slip is either displayed to the screen or not.	The Pay Slip is not displayed, and instead outputs the message “Invalid Choice” and moves on to the next employees.
21	char[]	cho	555	Stores the data into the variable to determine whether to display the pay slip or not.	The Pay Slip is either displayed to the screen or not.	The Pay Slip is not displayed, and instead outputs the message “Invalid Choice” and moves on to the next employees.
22	char[]	ano	Yes	Stores the data into the variable to determine whether to display the report or not.	The Report is displayed so the screen.	The Report is displayed so the screen.
23	char[]	ano	No	Stores the data into the variable to determine whether to	The Report is not displayed to the screen.	The Report is not displayed to the screen.

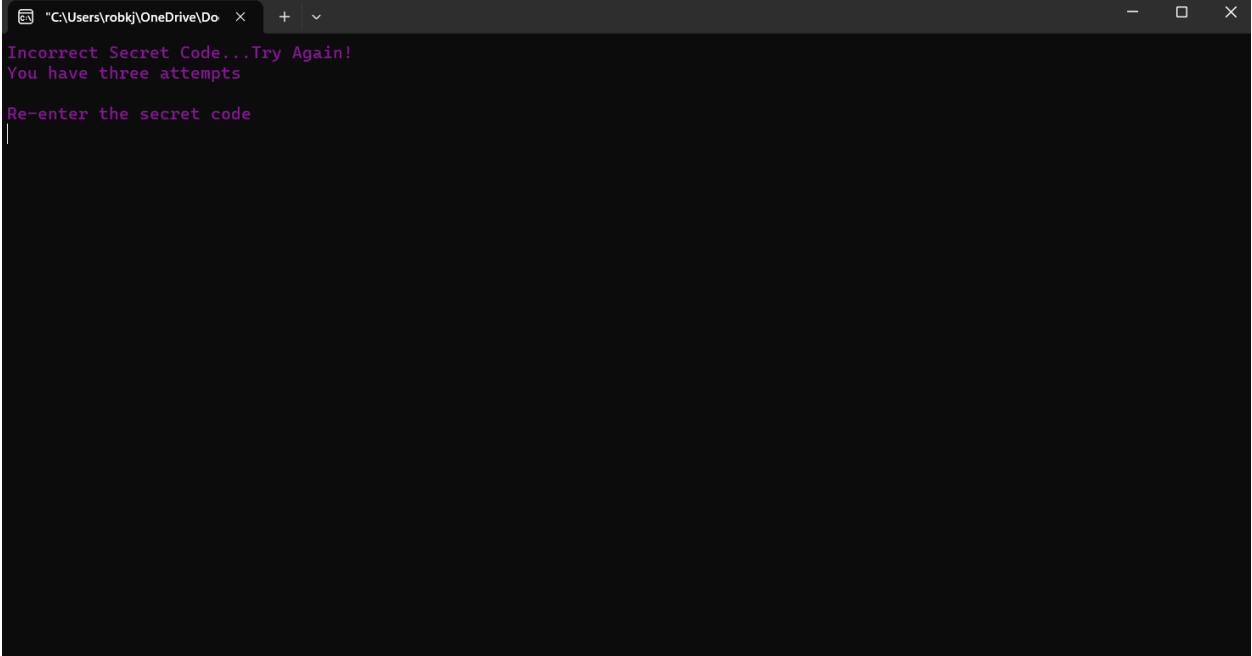
				display the report or not.		
24	char[]	ano	Maybe	Stores the data into the variable to determine whether to display the report or not.	The Report is either displayed to the screen or not.	The Report is not displayed, and instead takes the user back to the main menu.
25	char[]	ano	555	Stores the data into the variable to determine whether to display the report or not.	The Report is either displayed to the screen or not.	The Report is not displayed, and instead takes the user back to the main menu.
26	Integer	sel	1	Stores the data into the variable to determine whether to display employees, update employee, search employee, delete employee, add employee or return to main menu.	Gives the user the opportunity to display the employees from the file.	Gives the user the opportunity to display the employees from the file.
27	integer	sel	2	Stores the data into the variable to determine whether to display employees, update employee, search employee,	Gives the user the opportunity to update an employee's record and make changes to the file.	Gives the user the opportunity to update an employee's record and make changes to the file.

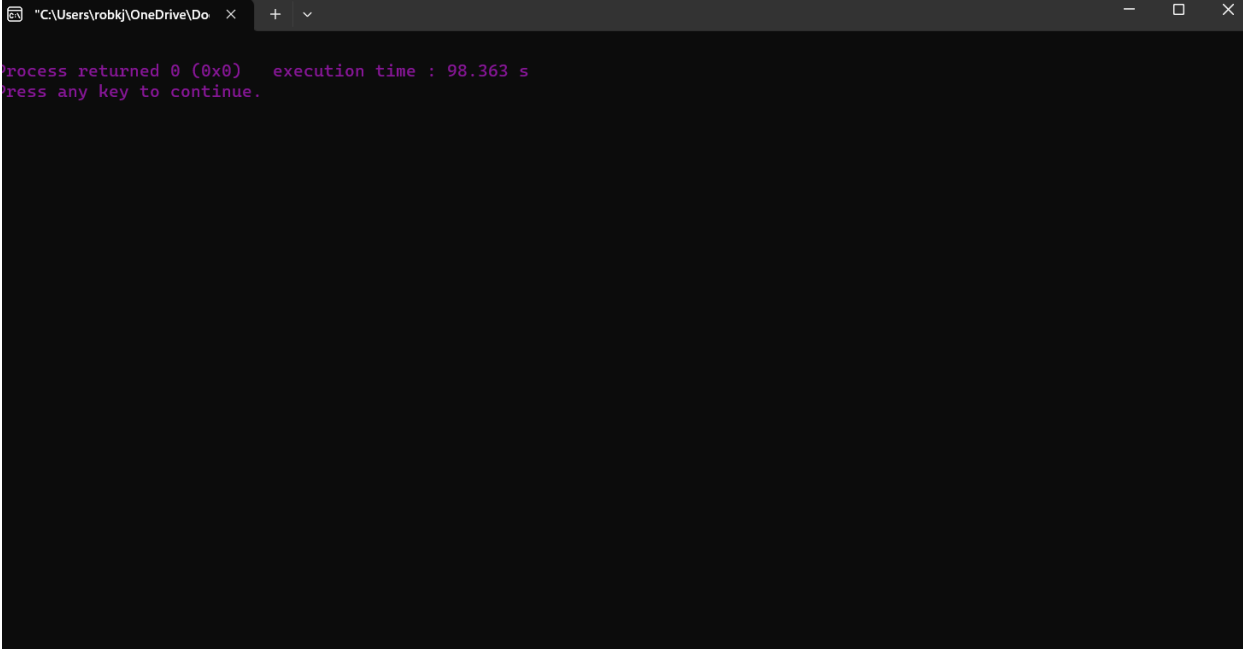
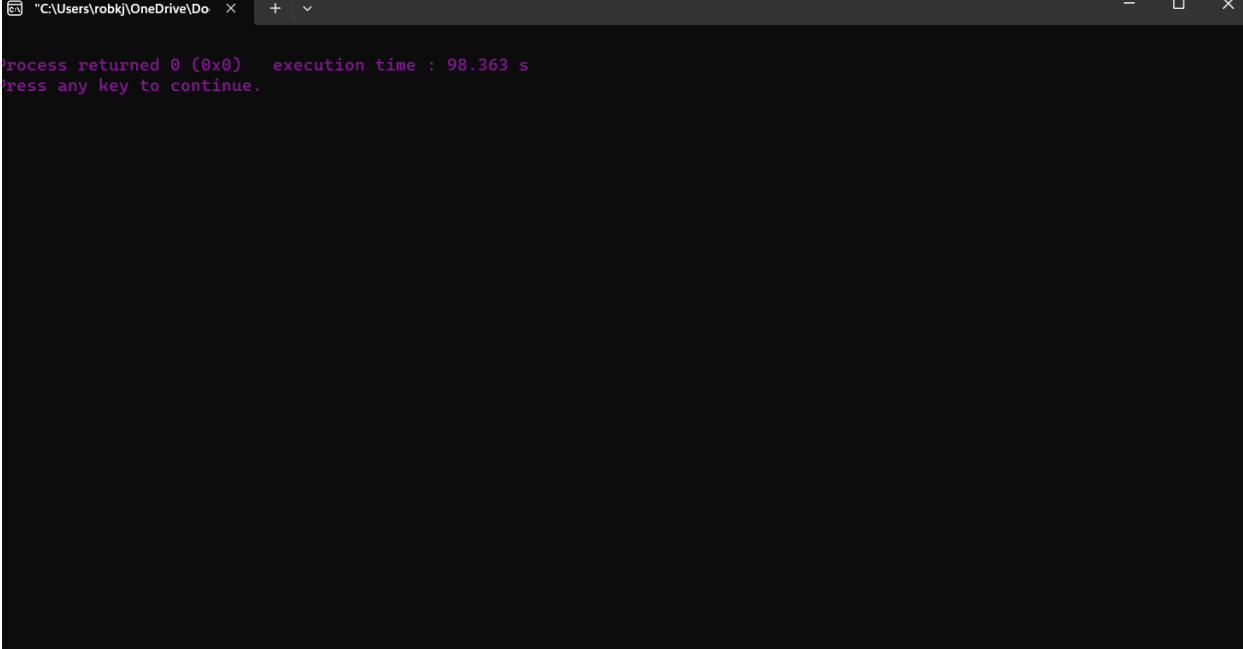
				delete employee, add employee or return to main menu.		
28	Integer	sel	3	Stores the data into the variable to determine whether to display employees, update employee, search employee, delete employee, add employee, or return to main menu.	Gives the user the opportunity to search for an employee's record and output its contents from a file.	Gives the user the opportunity to search for an employee's record and output its contents from a file.
29	Integer	sel	4	Stores the data into the variable to determine whether to display employees, update employee, search employee, delete employee, add employee or return to main menu.	Gives the user the opportunity to delete an employee's record and make the changes to the file.	Gives the user the opportunity to delete an employee's record and make the changes to the file.
30	integer	sel	5	Stores the data into the variable to determine whether to display	Gives the user the opportunity to add employees records and	Gives the user the opportunity to add employees records and

				employees, update employee, search employee, delete employee, add employee or return to main menu.	make changes to the file.	make changes to the file.
31	Integer	sel	0	Stores the data into the variable to determine whether to display employees, update employee, search employee, delete employee, add employee or return to main menu.	Takes the user back to the main menu.	Takes the user back to the main menu.
32	Integer	sel	7	Stores the data into the variable to determine whether to display employees, update employee, search employee, delete employee, add employee or return to main menu.	Gives the user the opportunity to either display, update, search, delete or add employees and make changes to the file.	Outputs the message “Invalid Option” and as a result takes the user back to the main menu.

33	Integer	sel	word	Stores the data into the variable to determine whether to display employees, update employee, search employee, delete employee, add employee or return to main menu.	Gives the user the opportunity to either display, update, search, delete or add employees and make changes to the file.	Outputs the message “Invalid Option” and as a result takes the user back to the main menu.
----	---------	-----	------	--	---	--

Test Results

Test #	Results
1	
2	

3	 The screenshot shows a Windows command prompt window with a single tab titled "C:\Users\robkj\OneDrive\Do". The window has standard Windows window controls (minimize, maximize, close) in the top right corner. The command prompt displays the following text in a monospaced font: "Process returned 0 (0x0) execution time : 98.363 s" followed by "Press any key to continue." on the next line. The rest of the window is black.
4	 This screenshot is identical to the one in row 3. It shows a Windows command prompt window with a tab titled "C:\Users\robkj\OneDrive\Do". The output text is "Process returned 0 (0x0) execution time : 98.363 s" followed by "Press any key to continue." on the next line. The window has standard Windows window controls in the top right corner.

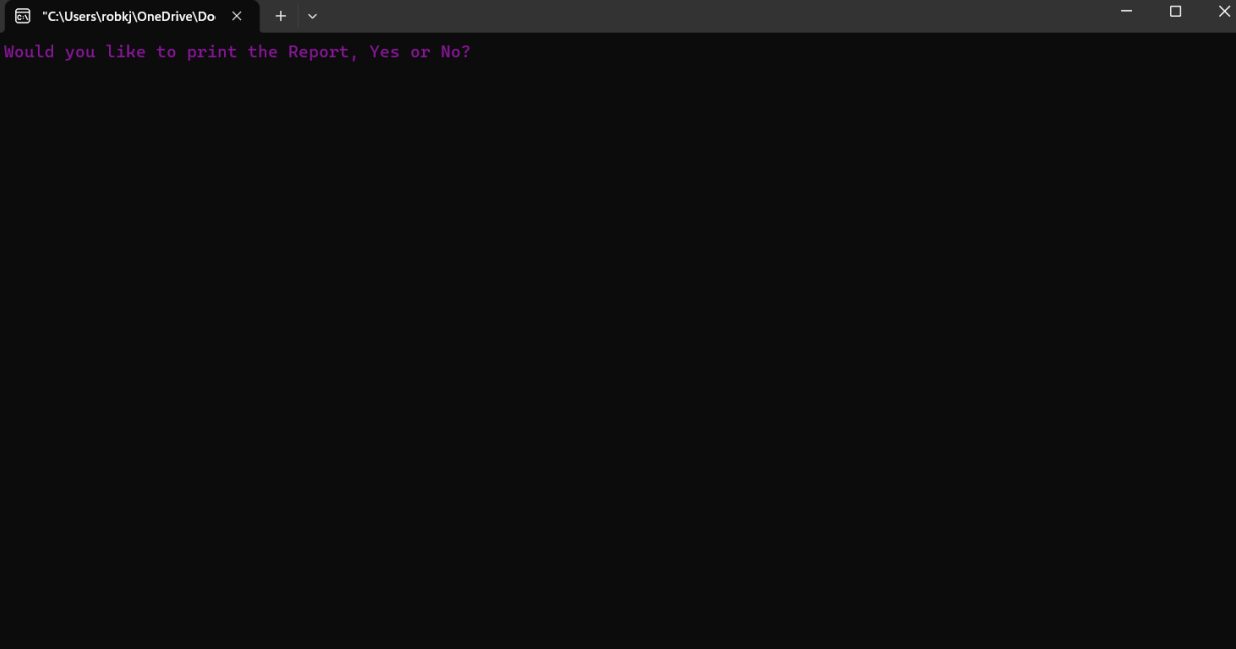
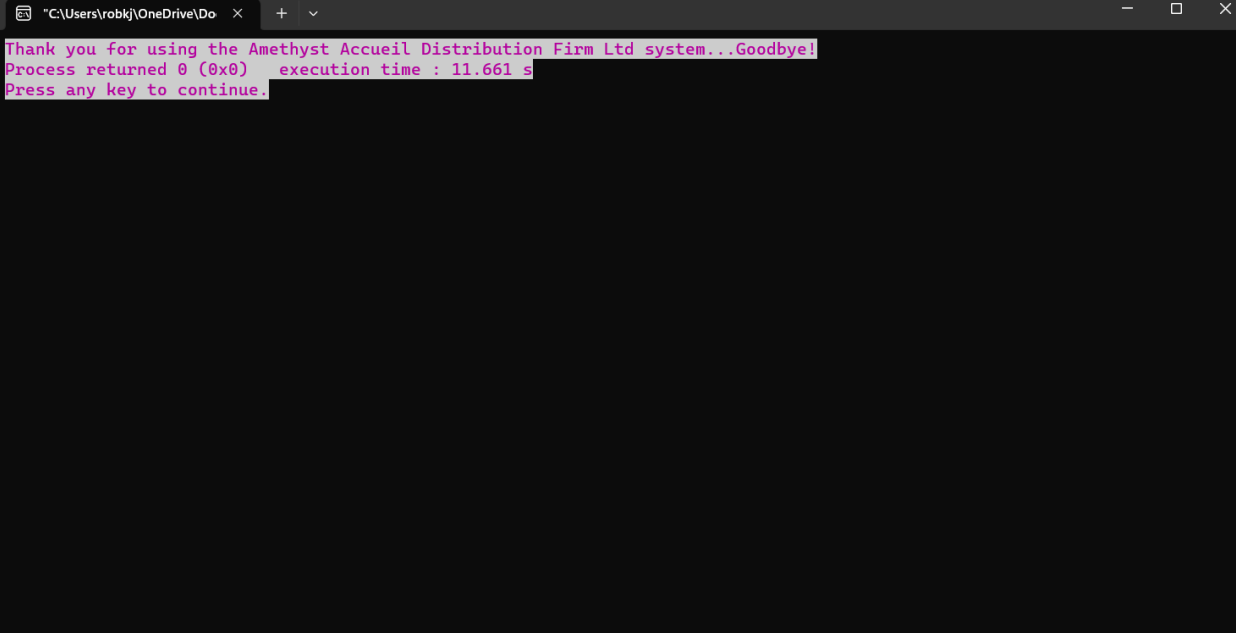
5

You've selected option 1: Manage Employees

6

```
C:\Users\robkj\OneDrive\Do  +  v
You've selected option 2: Print Pay Slip

Processing for Employee ID: 12
Enter the number of hours worked by the employee
```

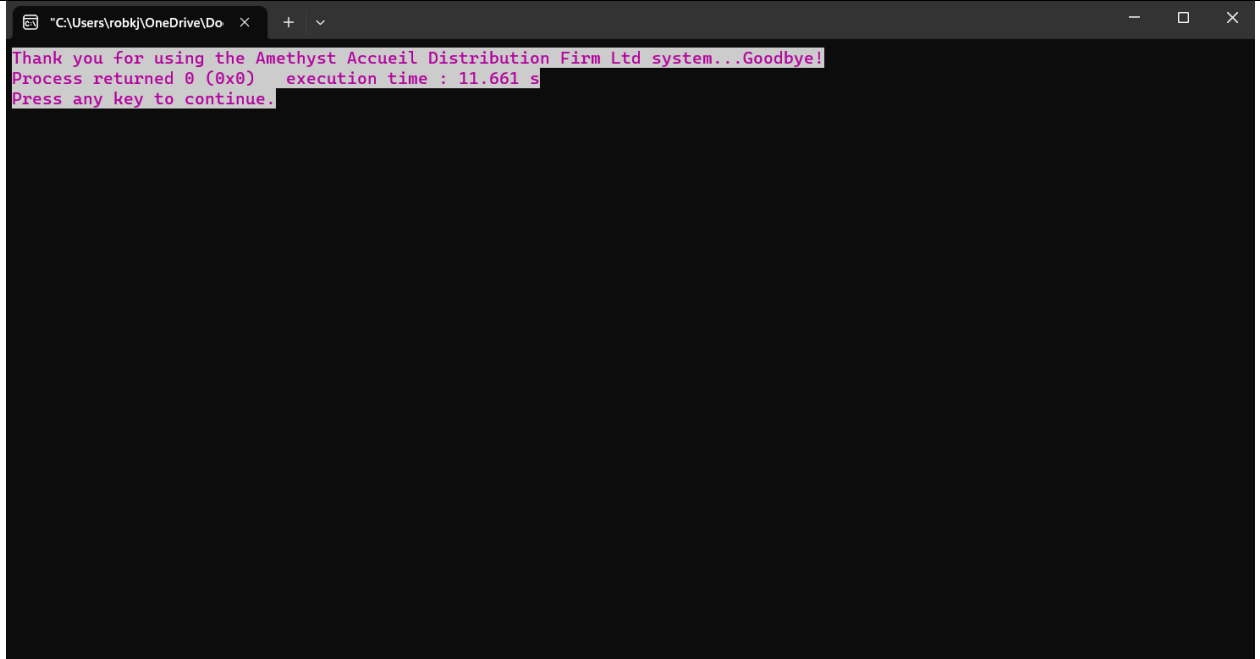
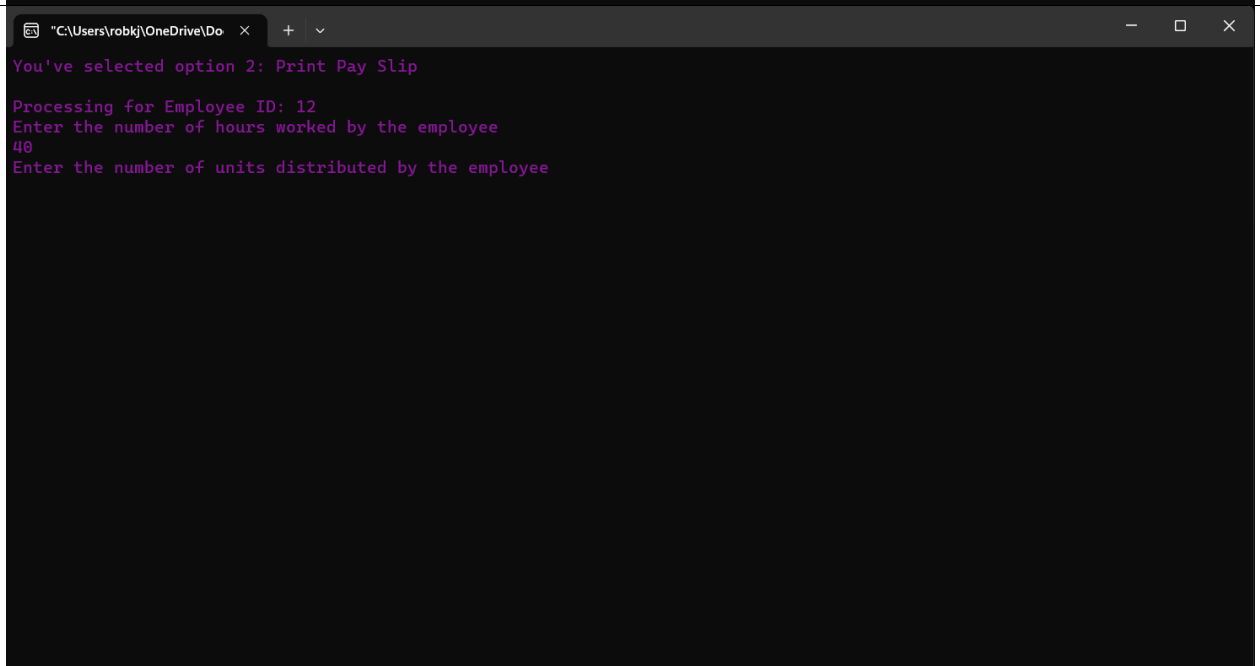
7	
8	

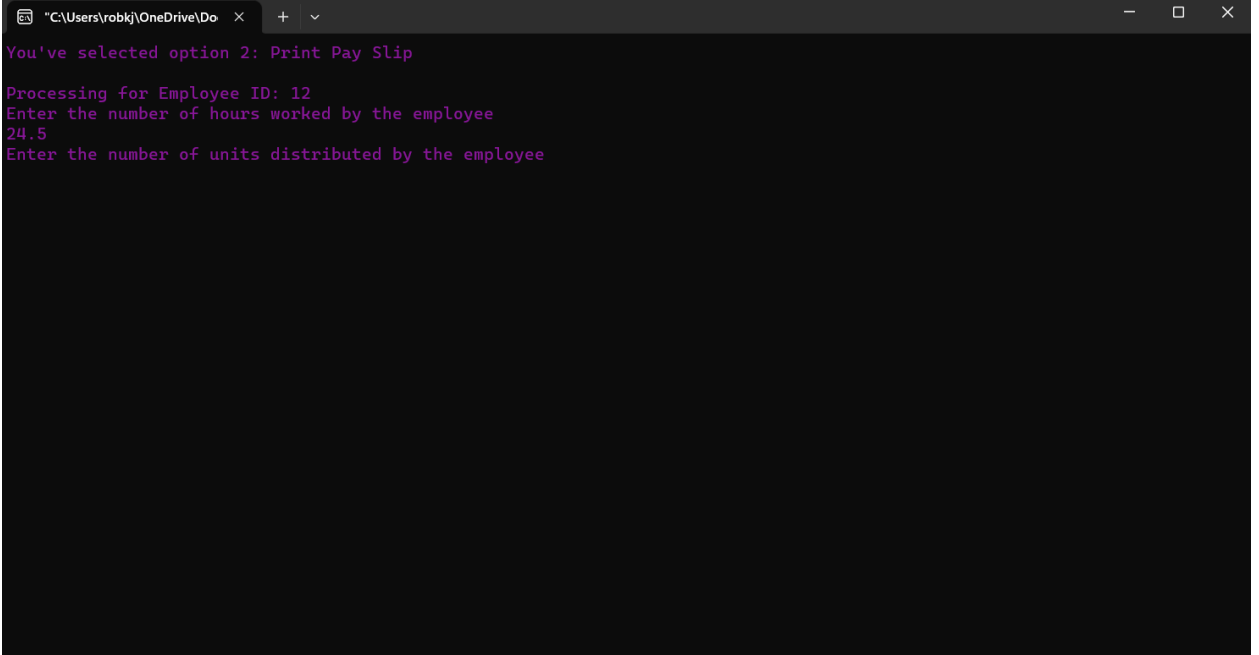
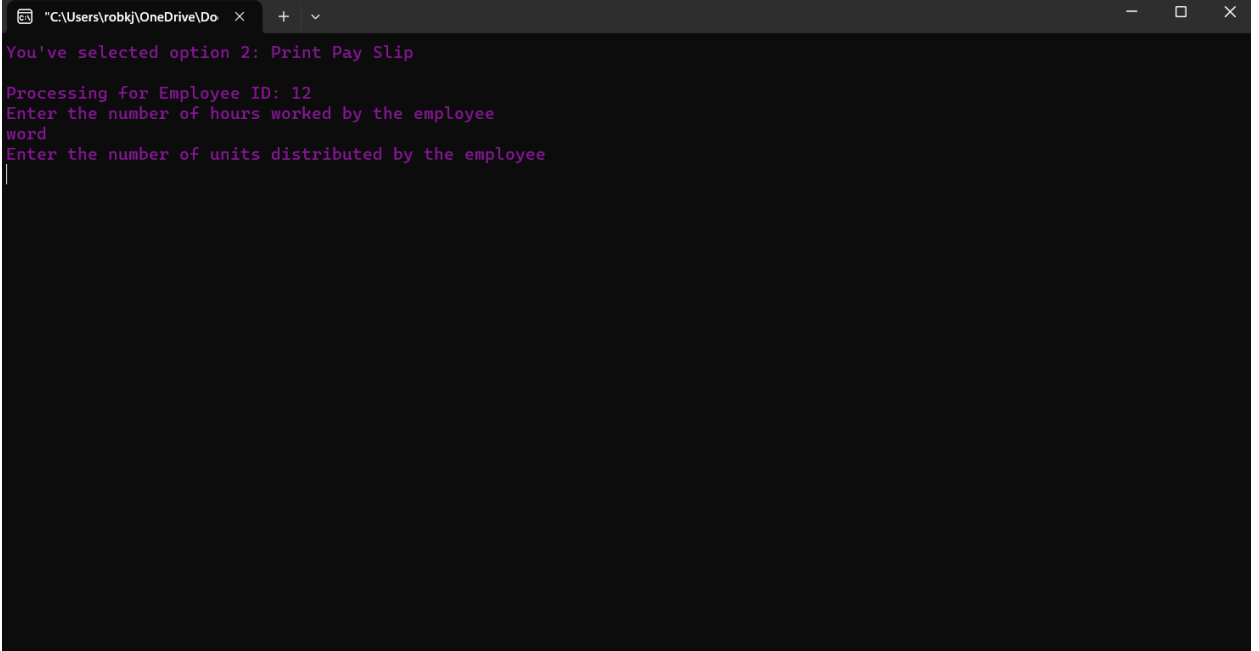
9

[illegible]

10

Thank you for using the Amethyst Accueil Distribution Firm Ltd system...Goodbye!
Process returned 0 (0x0) execution time : 11.661 s
Press any key to continue.

11	 The screenshot shows a Windows command prompt window with the title bar "C:\Users\robkj\OneDrive\Do". The text inside the window is as follows: <pre>Thank you for using the Amethyst Accueil Distribution Firm Ltd system...Goodbye! Process returned 0 (0x0) execution time : 11.661 s Press any key to continue.</pre>
12	 The screenshot shows a Windows command prompt window with the title bar "C:\Users\robkj\OneDrive\Do". The text inside the window is as follows: <pre>You've selected option 2: Print Pay Slip Processing for Employee ID: 12 Enter the number of hours worked by the employee 40 Enter the number of units distributed by the employee</pre>

13	 <pre>"C:\Users\robkj\OneDrive\Do x + v You've selected option 2: Print Pay Slip Processing for Employee ID: 12 Enter the number of hours worked by the employee 24.5 Enter the number of units distributed by the employee</pre>
14	 <pre>"C:\Users\robkj\OneDrive\Do x + v You've selected option 2: Print Pay Slip Processing for Employee ID: 12 Enter the number of hours worked by the employee word Enter the number of units distributed by the employee</pre>

15	A screenshot of a terminal window with a dark background. The title bar shows a file path: "C:\Users\robkj\OneDrive\Do". The terminal displays the text "Would you like to print the employee's pay slip, Yes or No?" in a light blue font.
16	A screenshot of a terminal window with a dark background. The title bar shows a file path: "C:\Users\robkj\OneDrive\Do". The terminal displays the text "Invalid Response...Proceeding...Press enter" in a light blue font.

17

A screenshot of a terminal window with a dark background and light blue text. The text reads: "Invalid Response...Proceeding...Press enter". The window title bar shows the path "C:\Users\robkj\OneDrive\Do" and standard window controls.

18

A screenshot of a terminal window with a dark background and light blue text. The text shows a pay slip for ID: 12. It includes a header for "Amethyst Accueil - Distribution Firm Ltd" and two tables: one for Earnings and one for Deductions.

Pay Slip being created...wait...

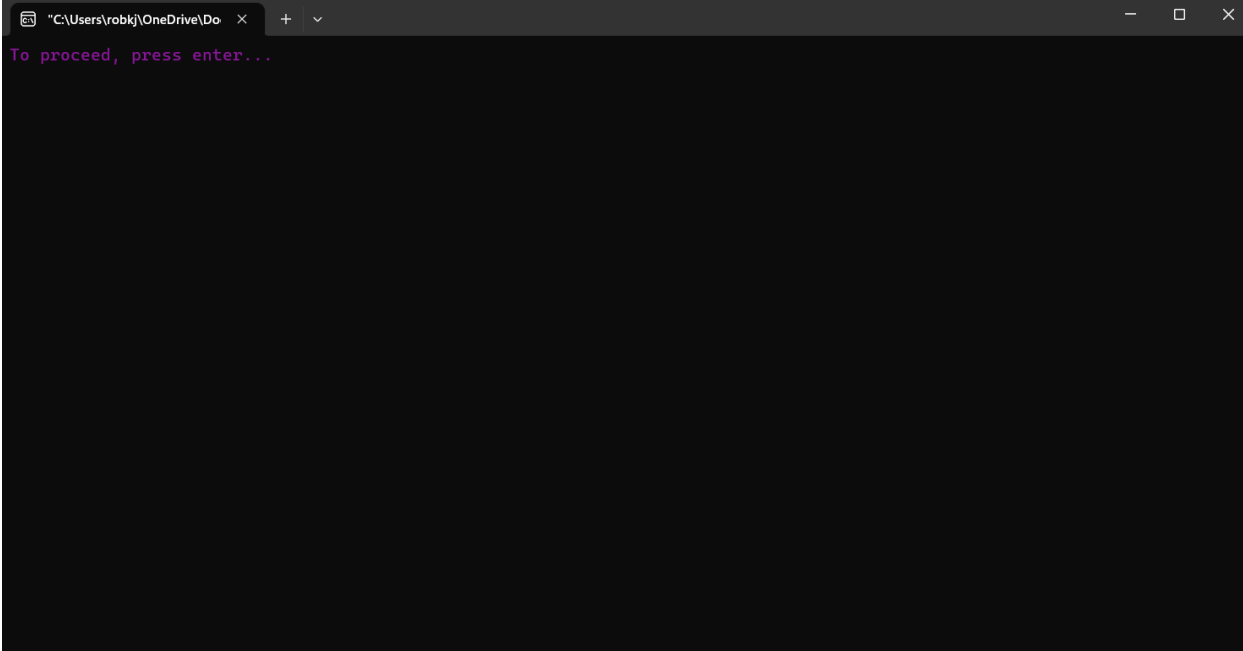
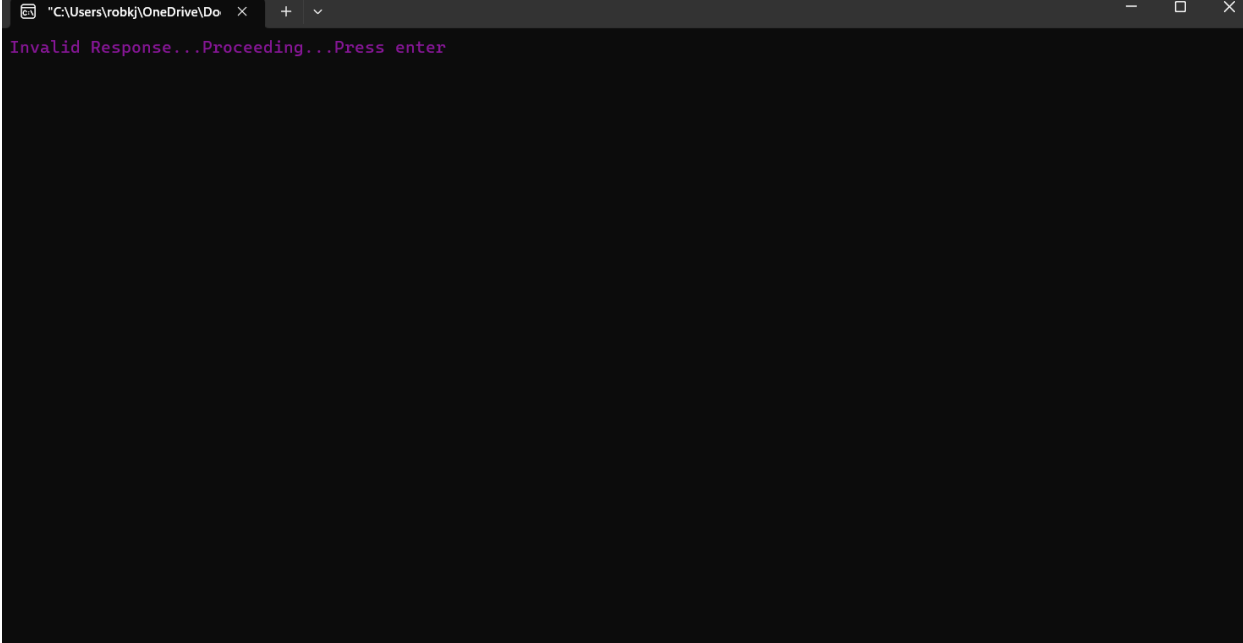
=====Amethyst Accueil - Distribution Firm Ltd=====

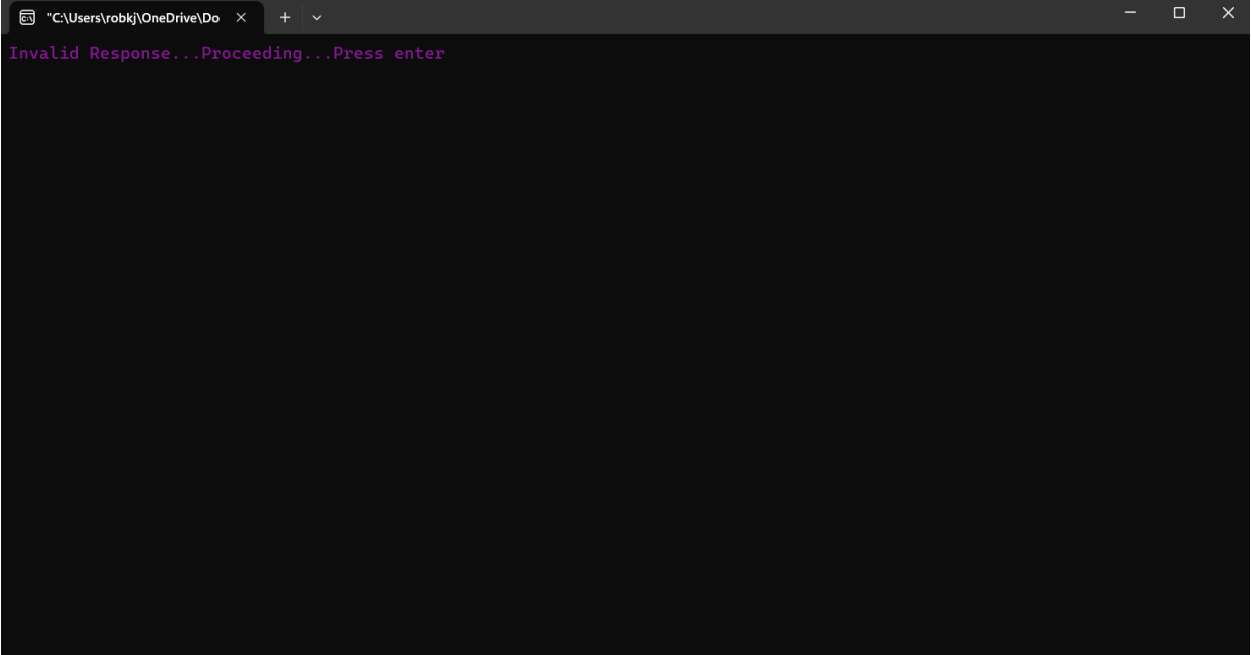
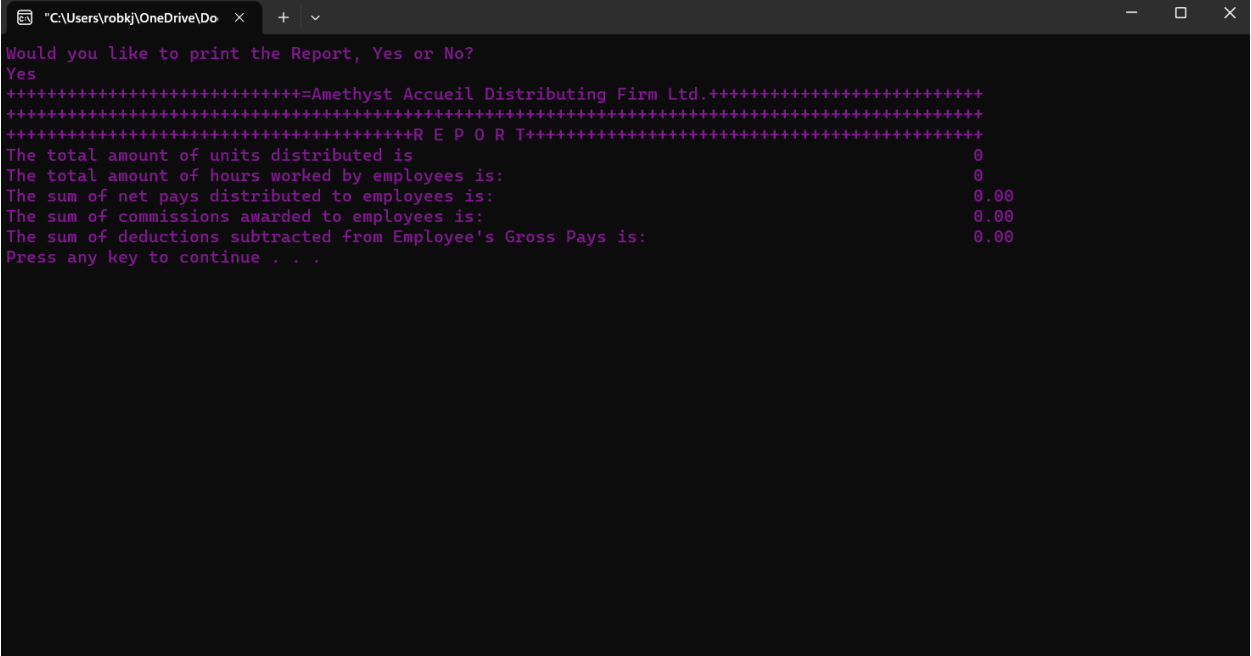
-----Pay Slip for ID: 12-----

Earnings	Hours	Rate	Current
Standard Pay	40	65.00	2600.00
Overtime Pay	0	1.50	0.00
Commission			50.00
Gross Pay			2650.00

=====

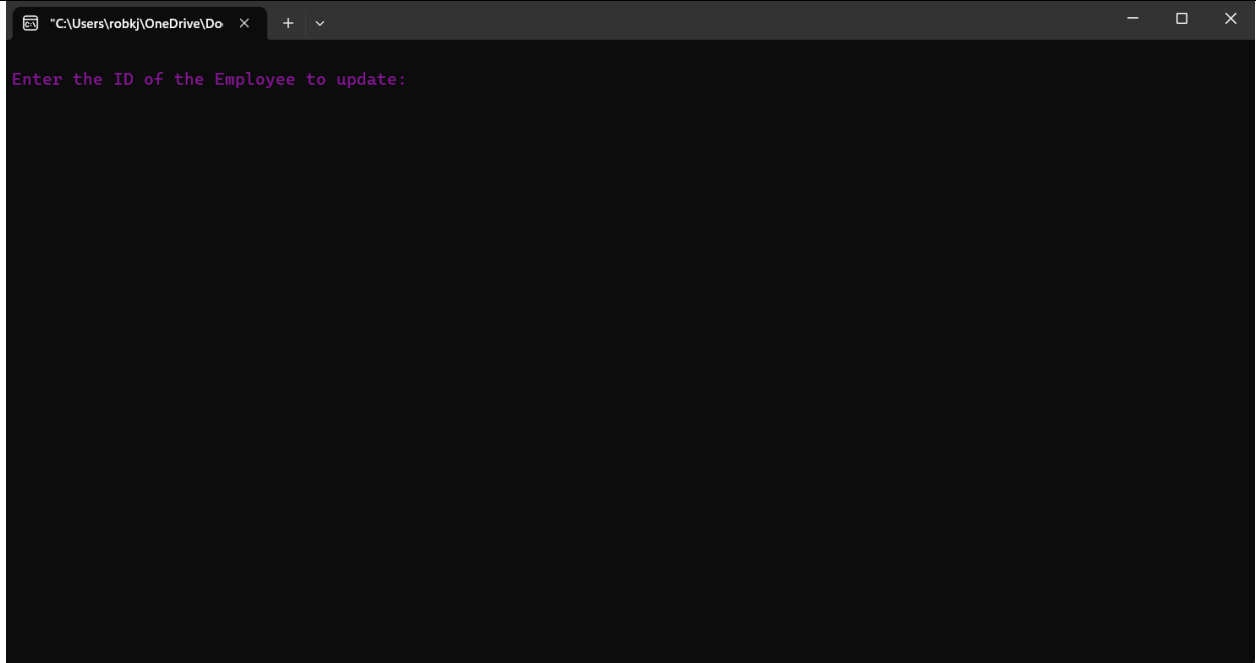
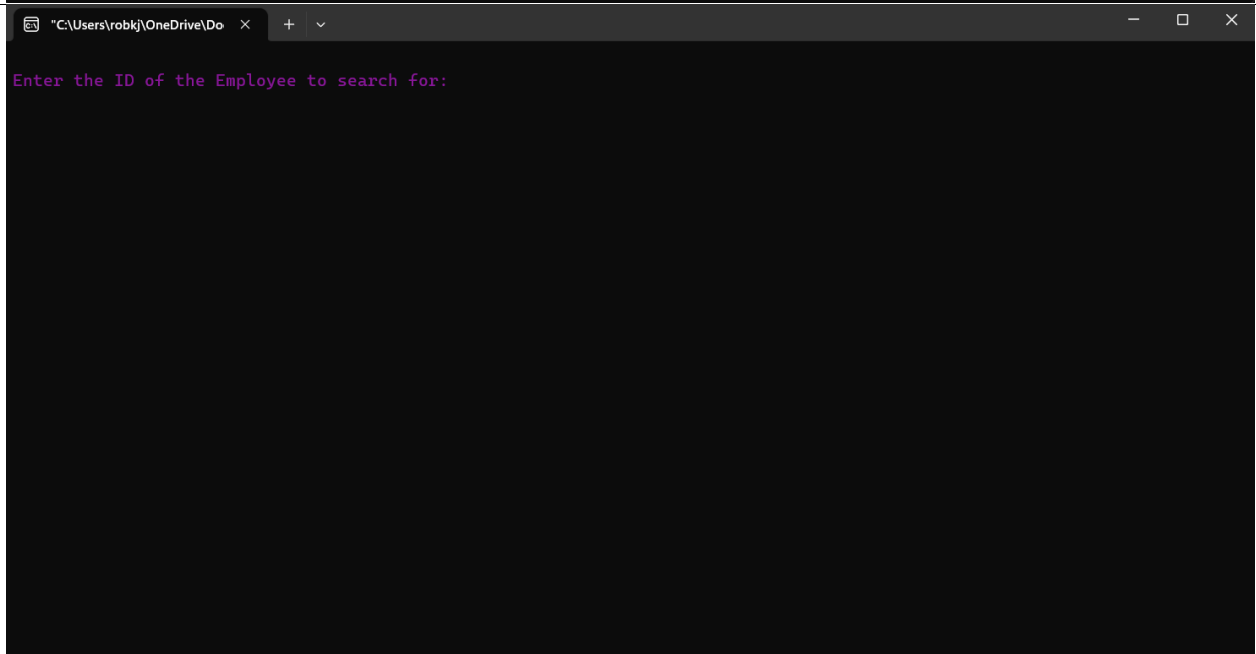
Deductions	Current
Social Security Tax	26.50
State Income Tax	185.50
NHT	106.00

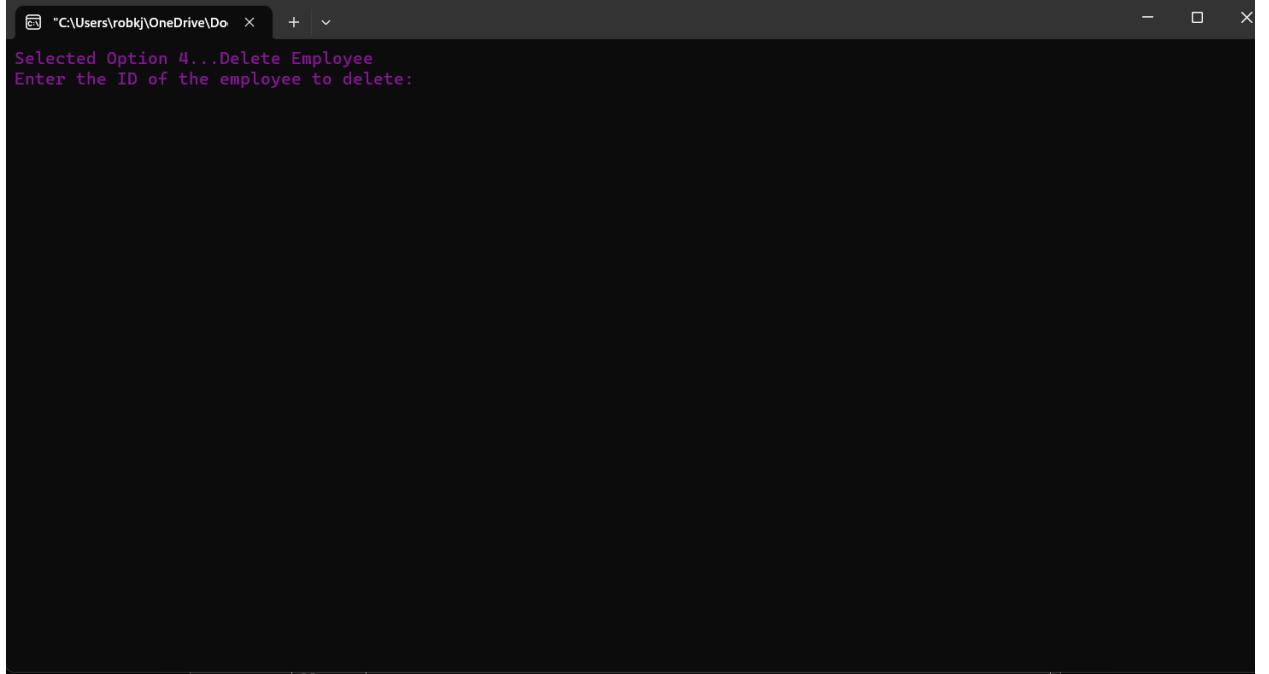
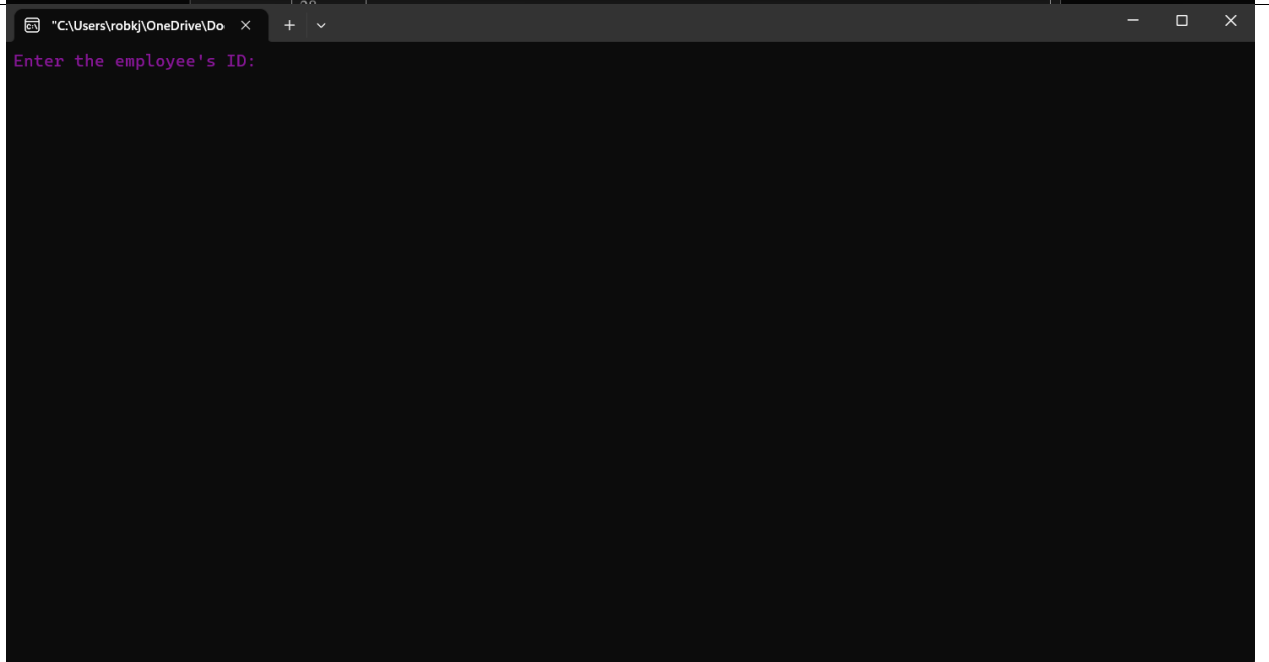
19	 A screenshot of a terminal window with a dark background. The title bar shows a single tab with the path "C:\Users\robkj\OneDrive\Do". The terminal displays the text "To proceed, press enter..." in a light purple font.
20	 A screenshot of a terminal window with a dark background. The title bar shows a single tab with the path "C:\Users\robkj\OneDrive\Do". The terminal displays the text "Invalid Response...Proceeding...Press enter" in a light purple font.

21	 A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\robkj\OneDrive\Do". The window contains the text "Invalid Response...Proceeding...Press enter" in a purple font.
22	 A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\robkj\OneDrive\Do". The window contains the following text in a purple font: <pre>Would you like to print the Report, Yes or No? Yes =====Amethyst Accueil Distributing Firm Ltd.===== =====R E P O R T===== The total amount of units distributed is 0 The total amount of hours worked by employees is: 0 The sum of net pays distributed to employees is: 0.00 The sum of commissions awarded to employees is: 0.00 The sum of deductions subtracted from Employee's Gross Pays is: 0.00 Press any key to continue . . .</pre>

23	<pre> You've been granted access -----Main Menu /--) /-- / /-- / /--) /-- / /-- / /-- / /-- / /-- / /-- / / (/ / / < < / < < / / / (/ / /) < < / ((((/ < / < < / / </pre> From the menu below, pick a number that corresponds with your desired option Manage Employees.....1 Print Pay Slip.....2 Print Report.....3 If you wish to exit the system, enter 0
24	<pre> You've been granted access -----Main Menu /--) /-- / /-- / /--) /-- / /-- / /-- / /-- / /-- / /-- / / (/ / / < < / < < / / / (/ / /) < < / ((((/ < / < < / / </pre> From the menu below, pick a number that corresponds with your desired option Manage Employees.....1 Print Pay Slip.....2 Print Report.....3 If you wish to exit the system, enter 0

25	<pre> You've been granted access -----Main Menu /--\ _/_/_ _/__ /--\ /--\ /--/ ----- /_/ /_ -- , -_ /_/ /--/ _ . . . _ o // / (_/ / / <-</-<_/_ /_/ (_/_/_)-<_ / (_(_(_(_/_/<-<-</- / From the menu below, pick a number that corresponds with your desired option Manage Employees.....1 Print Pay Slip.....2 Print Report.....3 If you wish to exit the system, enter 0 </pre>
26	<pre> Selected Option: 1...Display Employees Employee records: 5,Tamara,Williams,F,543216,876543219 7,Harry,Potter,M,654654,876876987 8,Robyn,Sterling,F,425367,132536527 16,Hermoine,Granger,F,76578,98767895 17,Rary,Tary,M,351725,162746829 19,Freddy,Male,M,716289,152793517 20,Rocky,Road,M,143526,162537492 21,Sasha,Sloan,F,91828,813826739 30,Baby,Bash,M,536378,153467381 35,Garfiled,Groans,M,625683,918635718 15,Abel,Tesfaye,M,76890,161491124 61,Kenya,Karly,F,90908,768918276 </pre>

27	 A screenshot of a terminal window with a dark background. The title bar shows a file path: "C:\Users\robkj\OneDrive\Do". The terminal displays the text "Enter the ID of the Employee to update:" in a light blue font.
28	 A screenshot of a terminal window with a dark background. The title bar shows a file path: "C:\Users\robkj\OneDrive\Do". The terminal displays the text "Enter the ID of the Employee to search for:" in a light blue font.

29	 A screenshot of a terminal window with a dark background. The title bar shows a single tab with the path "C:\Users\robkj\OneDrive\Do". The terminal text is as follows: <pre>Selected Option 4...Delete Employee Enter the ID of the employee to delete:</pre>
30	 A screenshot of a terminal window with a dark background. The title bar shows a single tab with the path "C:\Users\robkj\OneDrive\Do". The terminal text is as follows: <pre>Enter the employee's ID:</pre>

31

You've been granted access

_Main Menu

[illegible]

```

From the menu below, pick a number that corresponds with your desired option
Manage Employees.....1
Print Pay Slip.....2
Print Report.....3
If you wish to exit the system, enter 0

```

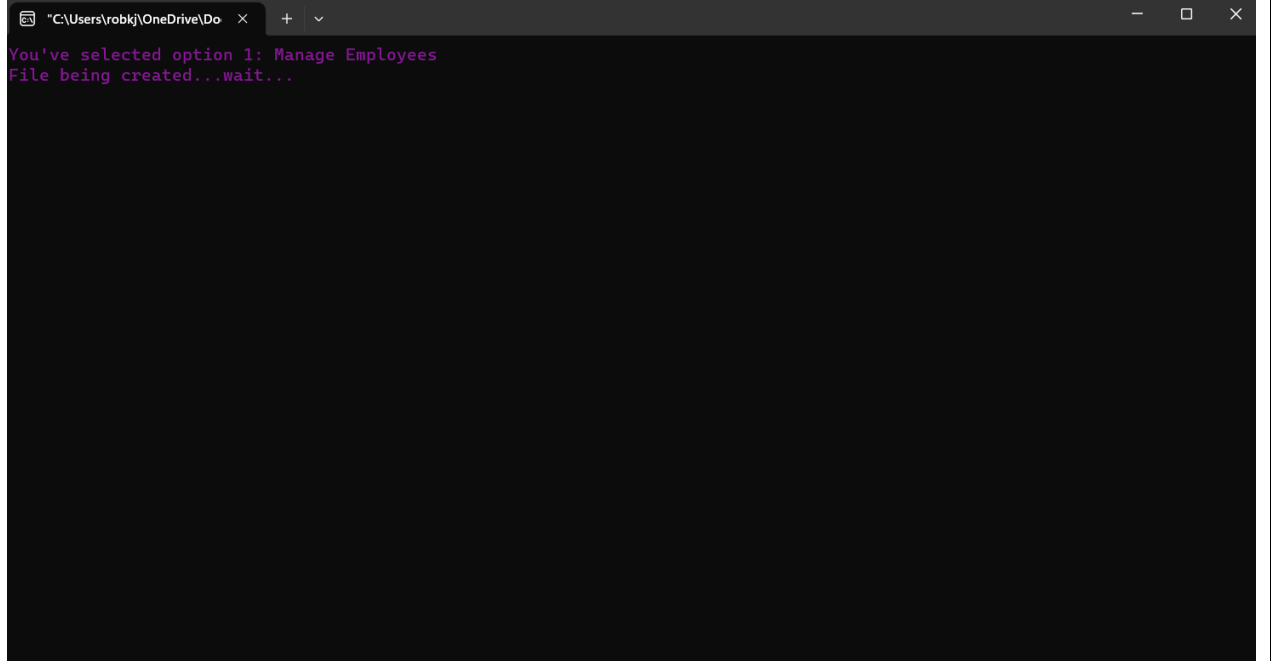
32

Invalid Choice

[illegible]

_Employee Menu

33



The screenshot shows a Windows File Explorer window with the address bar displaying "C:\Users\robkj\OneDrive\Do". The window contains two lines of text in a purple font: "You've selected option 1: Manage Employees" and "File being created...wait...". The window has a standard Windows title bar with minimize, maximize, and close buttons.

Error Trapping

Error trapping primarily focuses on handling file operations, input validation, and ensuring graceful program execution in various scenarios. Here are the areas where error trapping is implemented:

1. File Operations:

In functions like ``PAYSLIP``, ``report``, and ``employf``, file operations such as opening, reading, writing, and closing files are performed. Error trapping is implemented to handle cases where files cannot be opened or created.

- Example:

```
file = fopen ("employee.txt", "r");
if (file == NULL){
    printf ("An error occurred while opening the file. \n");
    return;
}
```

2. Input Validation:

- Input validation is crucial for ensuring the program handles unexpected or invalid user inputs gracefully.

- In functions like ``PAYSLIP``, ``report``, and ``employf``, user inputs are validated to ensure they are of the expected type and within acceptable ranges.

- Example:

```
printf("Enter the employee ID to search for: ");
scanf("%d", &search_num);
```

3. Function Return Values:

- Functions returning a value usually have error trapping to handle cases where the function fails to execute as expected.

- Example:

```
if (file4 == NULL) {
    printf("An error has occurred while opening file4\n");
    return;
}
```

4. Conditional Checks:

- Conditional checks are implemented to verify whether certain operations are successful or not, and actions are taken accordingly.

- Example:

```
if (found) {
    printf("Record updated successfully.\n");
} else {
    printf("Record doesn't seem to exist.\n");
    remove("copy.employee.txt"); // Remove the temporary file
}
```

5. Loop Condition Checking:

The loop condition in the for loop ensures that the loop terminates when the user exceeds the maximum number of attempts or successfully enters the correct password.

Example:

```
for (x = 1; x <= 3 && ent_pass != 5791; x++)
```

6. Error Message Printing:

Error messages are printed when the user enters an incorrect password or when the file operations encounter issues.

Example:

```
printf ("Incorrect Secret Code...Try Again!\n");
```

Correct Output

Correct Output Example 1: Wherein, the data added to the file is coherent with what is displayed on the program and as well as saved in the file.

```
11,Mekki,Mcdonald,F,56567,817283748
5,Tamara,Williams,F,543216,876543219
7,Harry,Potter,M,654654,876876987
8,Robyn,Sterling,F,425367,132536527
16,Hermoine,Granger,F,76578,98767895
17,Rary,Tary,M,351725,162746829
19,Freddy,Male,M,716289,152793517
20,Rocky,Road,M,143526,162537492
21,Sasha,Sloan,F,91828,813826739
30,Baby,Bash,M,536378,153467381
35,Garfiled,Groans,M,625683,918635718
15,Abel,Tesfaye,M,76890,161491124
61,Kenya,Karly,F,90908,768918276
```

```
Selected Option: 1...Display Employees
Employee records:
11,Mekki,Mcdonald,F,56567,817283748
5,Tamara,Williams,F,543216,876543219
7,Harry,Potter,M,654654,876876987
8,Robyn,Sterling,F,425367,132536527
16,Hermoine,Granger,F,76578,98767895
17,Rary,Tary,M,351725,162746829
19,Freddy,Male,M,716289,152793517
20,Rocky,Road,M,143526,162537492
21,Sasha,Sloan,F,91828,813826739
30,Baby,Bash,M,536378,153467381
35,Garfiled,Groans,M,625683,918635718
15,Abel,Tesfaye,M,76890,161491124
61,Kenya,Karly,F,90908,768918276
```

Correct Output Example 2: Wherein ID 11 worked a total of 56 hours that was inputted into the system, these hours were accurately calculated to print his Payslip, and coherent with that, the Payslip stored in the file.

```
|=====Amethyst Accueil - Distribution Firm Ltd=====|
|-----|
|-----Pay Slip for ID: 11-----|
|-----|
|Earnings |      Hours      |      Rate      |      Current
|Standard Pay |      40      |      65.00      |      2600.00
|Overtime Pay |      16      |      1.50      |      1560.00
|Commission  |              |                |      200.00
|Gross Pay   |              |                |      4360.00
|=====|
|Deductions |              |      Current
|Social Security Tax |              |      43.60
|State Income Tax |              |      305.20
|NHT |              |      174.40
|Pension Tax |              |      1090.00
|Insurance Tax |              |      130.80
|Total Deductions |              |      1744.00
|Net Pay |              |      2616.00
|-----|
|Press enter to proceed|
|
```

```
|=====Date: 2024-04-08|
|=====Amethyst Accueil - Distribution Firm Ltd=====|
|-----|
|-----Pay Slip for ID: 7-----|
|-----|
|Earnings |      Hours      |      Rate      |      Current
|Standard Pay |      40      |      65.00      |      2600.00
|Standard Pay |      56      |      65.00      |      2600.00
|Overtime Pay |      16      |      1.50      |      1560.00
|Commission  |              |                |      200.00
|Gross Pay   |              |                |      4360.00
|=====|
|Deductions |              |      Current
|Social Security Tax |              |      43.60
|State Income Tax |              |      305.20
|NHT |              |      174.40
|Pension Tax |              |      1090.00
|Insurance Tax |              |      130.80
|Total Deductions |              |      1744.00
|Net Pay |              |      2616.00
|-----|
```

Usability

Usability Example 1: Displays the message of “You’ve been granted access” to provide confirmation to the user of their access granted, followed by a header that displays the name of the firm in a large font size. The menu comprises of menu options alongside numbers to make the user experience easier and more efficient in lieu of typing out the entire menu option itself.

```

You've selected option 1: Manage Employees

  /--\      /--\      /--\      /--\      /--\
 /--/  /--/  /--/  /--/  /--/  /--/  /--/  /--/  /--/
/  ( /  /  /  /  /  /  /  /  /  /  /  /  /  /  /  /
/

-----Employee Menu-----

  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |

+-----+
| Select an option |
+-----+
| 1. Display Employees |
+-----+
| 2. Update Employee |
+-----+
| 3. Search for an Employee |
+-----+
| 4. Delete Employee |
+-----+
| 5. Add Employee |
+-----+
| 0. Return to the main menu |
+-----+

```

Usability Example 2: Similarly to usability example two remains consistent, to provide a cohesive set of prompts to the user, which makes the content more predictable and easier to maneuver around.

Conclusion

Amethyst Accueil Distribution Firm Ltd encountered significant issues with their system's key features, including malfunctioning functionalities like Display, Add, Delete, Update, and Search, as well as a restrictive six-employee limit. Additionally, problems with payslip and report functions arose, such as inaccessible filed content and limited payslip generation.

However, strategic improvements have resulted in a much more effective and efficient system. The previously faulty features now operate seamlessly, and the employee limit has been eliminated to accommodate the firm's growth. Payslip and report functions have been enhanced for accessibility and flexibility.

Looking forward, embracing technological advancements in graphic design and programming can further improve the system's user experience and functionality. Suggestions for improvement include introducing emergency management and inventory functions and implementing network integration for enhanced data sharing.

By continually adapting to technological advancements, Amethyst Accueil Distribution Firm Ltd can ensure its system remains robust and capable of meeting evolving user needs.

Appendix

Questionnaire for Employees

1. What is your age?
2. How long have you been operating as an employee for the Amethyst Accueil company?
3. On a scale of 1 to 10, how would you rate the clarity and ease of understanding the new digital payroll system.
4. Previously, Amethyst Accueil was purely paper based in recording and distribution on information, data, and records, as of recently, their system has adopted a partial digital system. With this knowledge, do you consider the information to be of a more accurate and premium quality? (Yes or No)
5. In relation to the previous question, please give a reason as to why this is answer.
6. How satisfied are you with the accessibility of your digital payroll system? If yes, please specify.
7. Do you find that the software is aesthetically pleasing to the eyes?
8. Are there any features in the digital payroll system that you find particularly beneficial or challenging?
9. Are there specific areas in which you believe additional training or support would be beneficial for a smoother transition?
10. Have you experienced any disruptions in accessing the digital payroll system, and if so, how were these issues resolved?
11. In your opinion, what steps could be taken to enhance employee engagement and understanding of the digital payroll system?
12. Please state any other concerns/queries and such that you may have as it relates to the current system.

Thank you for taking your time to fill out the questionnaire. Your responses are much valued.

Amethyst Accueil.

Interview for Management

1. In what ways has the digital system improved decision-making and strategic planning related to financial management?
2. How do you assess the accuracy and reliability of the data generated by the digital payroll system compared to the previous paper-based system?
3. Can you discuss any strategies for continuous improvement and innovation in the use of the digital system for payroll and employee record management?
4. How has the digital system adapted to changes in organizational structure, workforce size, or regulatory requirements, ensuring continued effectiveness?
5. Can you discuss specific objectives related to employee record management that the organization aims to achieve through the digital system?

Thank you for taking your time to answer this interview. Your responses are much valued.

Amethyst Accueil.